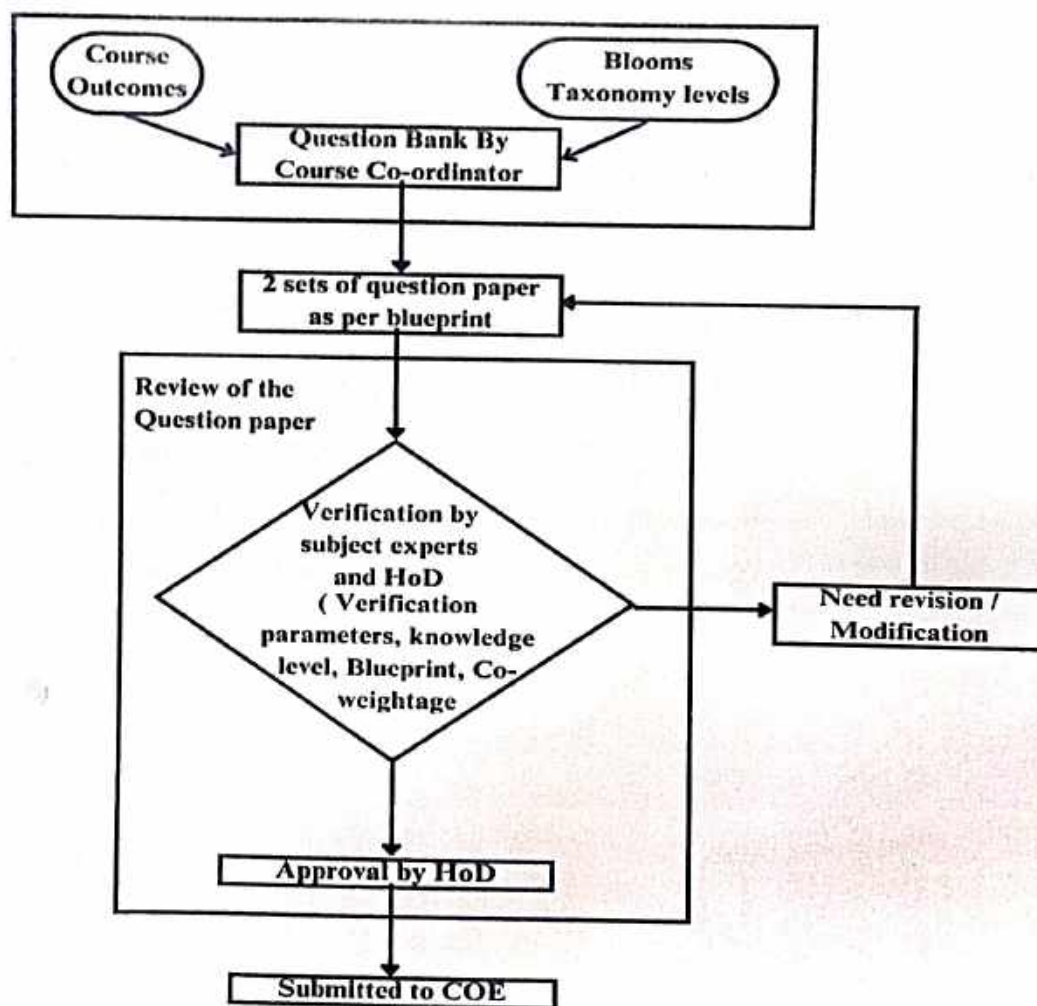




S.A ENGINEERING COLLEGE, CHENNAI - 77

(Autonomous - Institute Level Research Centre, Affiliated to Anna University)
Department of Computer Science and Engineering

PROCESS FOR INTERNAL ASSESSMENT QUESTION PAPER SETTINGS AND EVALUATION AND EFFECTIVE PROCESS IMPLEMENTATION



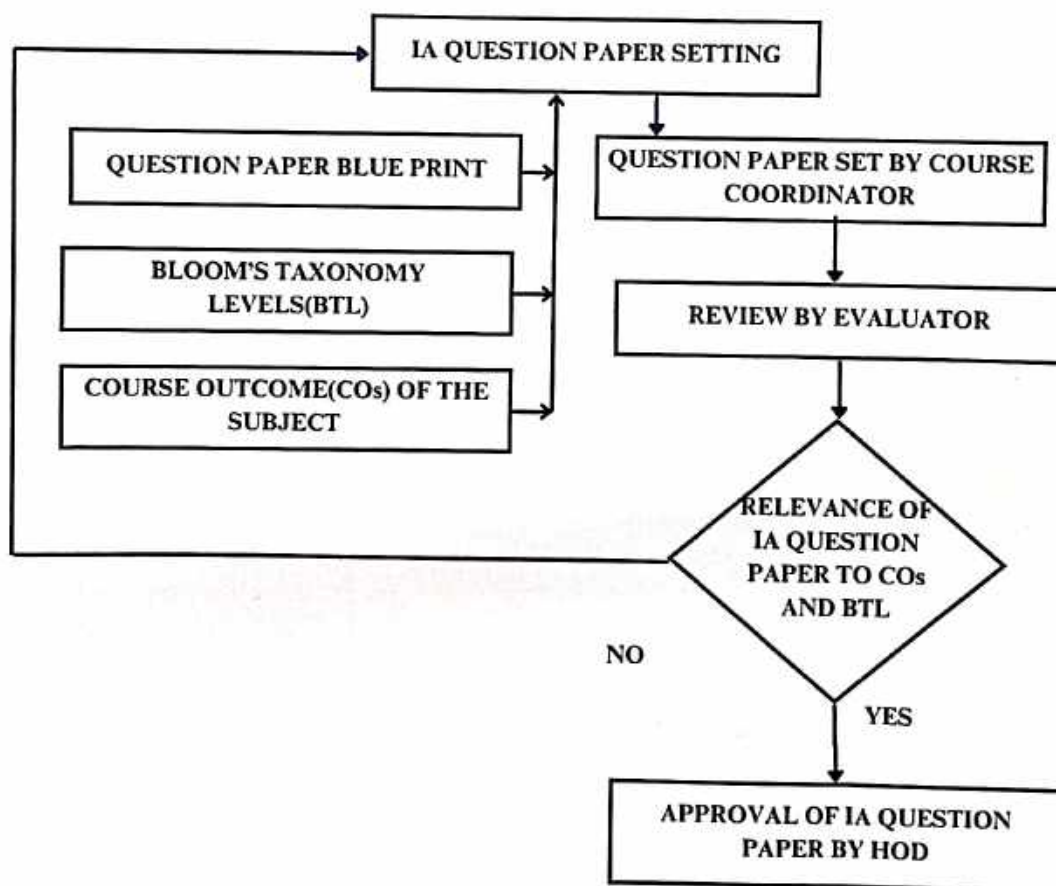


S.A ENGINEERING COLLEGE, CHENNAI - 77

(Autonomous - Institute Level Research Centre, Affiliated to Anna University)

Department of Computer Science and Engineering

PROCESS FOR QUALITY OF INTERNAL ASSESSMENT QUESTION PAPER



P.10 – Records of quality assessment in continuous internal evaluation-internal question paper, assignment, quiz, etc.

1. Records of Quality Assessment in Internal Evaluation

A. Process for Internal Question Paper Preparation and Evaluation

At the beginning of every semester, the Controller of Examinations (CoE) issues a circular to all departments to prepare question banks for all theory courses offered. Each Course Coordinator (CC) organizes a meeting with the course handling faculty to finalize:

- Number and type of questions to be prepared under each unit/topic.
- Bloom's Taxonomy Levels (BTL) for each question.
- Course Outcomes (COs) and Program Outcomes (POs) to be covered.

The question bank covers all five units of the course, ensuring a balance across cognitive levels (BTL 1–6). After internal verification by the Course Coordinator, it is submitted to the Department Evaluation Committee for validation against the Course Outcome Attainment Plan. Once approved, it is forwarded to the CoE for secure storage and usage in internal assessments.

Each semester includes two Internal Assessment (IA) examinations, following a prescribed institutional template (IA–I and IA–II), ensuring uniformity and outcome alignment.

B. Evolution of Internal Assessment Structure

Earlier Evaluation Pattern (Prior to R2020A)

Initially, the institution followed a **three-internal-assessment model**:

- **IA–I and IA–II:** 50 marks each, covering **two units** per test.
- **IA–III:** Comprehensive assessment for **all five units**.

The **best two out of three** scores were considered for internal marks computation.

However, during faculty and academic audit reviews, it was observed that this model created uneven weightage across courses and inconsistent focus on continuous improvement. To ensure uniform coverage, equal weightage and a balanced evaluation approach, the system was restructured under **R2020A** as follows.

R2020A Regulation

Particulars	IA–I	IA–II
Total Marks	100	100
Duration	3 Hours	3 Hours
Question Paper Pattern	Part A: $10 \times 2 = 20$ Marks Part B: $5 \times 13 = 65$ Marks Part C: $1 \times 15 = 15$ Marks	Part A: $10 \times 2 = 20$ Marks Part B: $5 \times 13 = 65$ Marks Part C: $1 \times 15 = 15$ Marks

Coverage	Unit 1,2 and 3(first half).	Unit 3(2 nd half),4 and 5.
Course Outcomes Covered	CO1, CO2, CO3	CO3, CO4, CO5
Bloom's Taxonomy Levels	BTL 1–4 (Remember, Understand, Apply, Analyse)	BTL 1–4 (Remember, Understand, Apply, Analyse)
Verification & Approval	Subject Expert & HoD	Subject Expert & HoD

Key Improvement:

- Two IAs ensure equal unit coverage (2.5 units per test) and uniform weightage across the semester.
- Encourages continuous learning rather than selective preparation for one best exam.

R2024 Regulation (Revised Pattern)

Under R2024, the internal assessment structure was further refined based on stakeholder feedback, student performance analytics, and faculty review.

Particulars	IA–I	IA–II
Total Marks	100	100
Duration	3 Hours	3 Hours
Pattern	Part A: $10 \times 2 = 20$ Marks Part B: $5 \times 16 = 80$ Marks	Part A: $10 \times 2 = 20$ Marks Part B: $5 \times 16 = 80$ Marks
Coverage	Unit 1,2 and 3(first half).	Unit 3(2 nd half),4 and 5.
Course Outcomes Covered	CO1, CO2, CO3	CO3, CO4, CO5
Bloom's Taxonomy Levels	BTL 1–4 (Remember, Understand, Apply, Analyse)	BTL 1–4 (Remember, Understand, Apply, Analyse)
Verification & Approval	Subject Expert & HoD	Subject Expert & HoD

Major Revision:

- Part C (15-mark question) was removed.
- The modified structure ensures clear alignment with revised CO–PO–BTL mapping and attainment targets defined in the OBE framework.

C. Internal Question Paper Quality Assurance Process

Step	Activity	Responsible Authority	Record Maintained
1	Preparation of Question Bank (CO & BTL based)	Course Coordinator & Faculty	Question Bank File
2	Preparation of IA-I & IA-II Papers (as per template)	Course Faculty	IA-I & IA-II Papers
3	Review and Verification	Subject Expert / HoD	Verification Form
4	Approval by Department Committee	HoD	Approved Copy
5	Submission to CoE	Course Coordinator	Sealed Copy
6	Post-exam Evaluation and Feedback	Faculty / Students	Evaluated Scripts, Feedback Report

S.A Engineering College, Chennai-77.
Department of Computer Science & Engineering
Rubrics for Internal Assessment

Degree/Branch/Class:
 Register No.

Academic Year/Semester:
 Subject Code/Subject Title:

Name of the Faculty:

1. Rubric for 2-Mark Questions (10 Questions)

Criteria	Excellent (2)	Good (1)	Poor (0)
Clarity of Concept	Accurate and precise answer	Partially correct or vague	Incorrect/No answer
Completeness	Complete response	Partially complete	Incomplete/No attempt

2. Rubric for 13-Mark Questions (5 Questions)

Criteria	Excellent (11-13)	Good (8-10)	Average (4-7)	Poor (0-3)
Understanding of Concept	Comprehensive explanation with strong reasoning	Good explanation with minor gaps	Basic explanation with limited reasoning	Incorrect/No understanding
Relevance & Accuracy	Highly relevant and accurate	Mostly relevant with minor errors	Some relevance but several errors	Irrelevant/Incorrect answer
Organization & Structure	Well-structured, logical flow	Clear structure with minor lapses	Some structure but inconsistent	Poorly organized/No structure
Use of Examples/Diagrams	Appropriate and accurate usage	Relevant but partially correct	Minimal or unclear	Not used/Incorrect

3. Rubric for 16-Mark Question (1 Question)

Criteria	Excellent (13-16)	Good (9-12)	Average (5-8)	Poor (0-4)
Depth of Knowledge	Extensive coverage with critical insight	Good coverage with minor gaps	Basic coverage, lacks depth	Inadequate/Incorrect
Analytical Ability	Excellent analysis and reasoning	Good reasoning with minor flaws	Limited analysis, superficial	No analysis/Incorrect
Completeness	Answer is complete and thorough	Mostly complete with minor omissions	Partially complete	Incomplete/No attempt
Presentation & Organization	Well-structured, clear and concise	Organized with minor issues	Somewhat disorganized	Poorly presented

S.A. ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University)

Accredited by NBA & NAAC with 'A' Grade

ISO 9001:2015 Certified Institution

Register No:

INTERNAL ASSESSMENT- I

ODD SEMESTER, 2024- 2025

DEPARTMENT OF _____

(Common to _____)

SUB CODE-SUB NAME

Time: 180 mins

COURSE OBJECTIVES:

•
•
•

COURSE OUTCOMES:

SUB CODE.1 :

SUB CODE.2 :

SUB CODE.3 :

SEMESTER-

MAXIMUM MARKS: 100

		CO.No.	BTL	Marks
Answer ALL the Questions PART - A (10 x 2 = 20 Marks)				
1		CO1	BTL-	2
2		CO1	BTL-	2
3		CO1	BTL-	2
4		CO1	BTL-	2
5		CO2	BTL-	2
6		CO2	BTL-	2
7		CO2	BTL-	2
8		CO2	BTL-	2
9		CO3	BTL-	2
10		CO3	BTL-	2
PART - B (5 x 13 = 65 Marks)				
11. (a)		CO1	BTL-	
(Or)				
11. (b)		CO1	BTL-	
12. (a)		CO1	BTL-	
(Or)				
12. (b)		CO1	BTL-	
13. (a)		CO2	BTL-	
(Or)				

13.(b)		CO2	BTL-	
14.(a)		CO2	BTL-	
(Or)				
14.(b)		CO2	BTL-	
15.(a)		CO3	BTL-	
(Or)				
15.(b)		CO3	BTL-	
PART – C (1 x 15 = 15 Marks)				
16.(a)			BTL-	
(Or)				
16.(b)			BTL-	

BTL-1 – Remember, BTL-2 – Understand, BTL-3 – Apply, BTL- 4–Analyze, BTL-5 – Evaluate,
BTL-6 – Create

COURSE COORDINATOR

EVALUATOR

HOD

S.A. ENGINEERING COLLEGE (An Autonomous Institution, Affiliated to Anna University) Accredited by NBA & NAAC with 'A' Grade ISO 9001:2015 Certified Institution		Register No:		
INTERNAL ASSESSMENT- II ODD SEMESTER, 2024- 2025 DEPARTMENT OF _____ (Common to _____) SUB CODE-SUB NAME _____ SEMESTER- _____				
Time: 180 mins COURSE OBJECTIVES: . . . COURSE OUTCOMES: SUB CODE.3 : SUB CODE.4 : SUB CODE.5 :		MAXIMUM MARKS: 100		
		CO.No.	BTL	Marks
Answer ALL the Questions PART – A (10 x 2 = 20 Marks)				
1		CO3	BTL-	2
2		CO3	BTL-	2
3		CO4	BTL-	2
4		CO4	BTL-	2
5		CO4	BTL-	2
6		CO4	BTL-	2
7		CO5	BTL-	2
8		CO5	BTL-	2
9		CO5	BTL-	2
10		CO5	BTL-	2
PART – B (5 x 13 = 65 Marks)				
11. (a)		CO3	BTL-	
(Or)				
11. (b)		CO3	BTL-	
12.(a)		CO4	BTL-	
(Or)				
12.(b)		CO4	BTL-	
13.(a)		CO4	BTL-	
(Or)				

13.(b)		CO4	BTL-	
14.(a)		CO5	BTL-	
(Or)				
14.(b)		CO5	BTL-	
15.(a)		CO5	BTL-	
(Or)				
15.(b)		CO5	BTL-	
PART – C (1 x 15 = 15 Marks)				
16.(a)			BTL-	
(Or)				
16.(b)			BTL-	

BTL-1 – Remember, BTL-2 – Understand, BTL-3 – Apply, BTL- 4–Analyze, BTL-5 – Evaluate,
BTL-6 – Create

COURSE COORDINATOR

EVALUATOR

HOD

S.A. ENGINEERING COLLEGE (An Autonomous Institution, Affiliated to Anna University) Accredited by NBA & NAAC with 'A' Grade ISO 9001:2015 Certified Institution		Register No:		
INTERNAL ASSESSMENT- I EVEN SEMESTER, 2024- 2025 DEPARTMENT OF _____ (Common to _____) SUB CODE-SUB NAME _____				
Time: 180 mins COURSE OBJECTIVES: • • • COURSE OUTCOMES: SUB CODE.1 : SUB CODE.2 : SUB CODE.3 :		SEMESTER-	MAXIMUM MARKS: 100	
		CO.No.	BTL	Marks
Answer ALL the Questions PART – A (10 x 2 = 20 Marks)				
1		CO1	BTL-	2
2		CO1	BTL-	2
3		CO1	BTL-	2
4		CO1	BTL-	2
5		CO2	BTL-	2
6		CO2	BTL-	2
7		CO2	BTL-	2
8		CO2	BTL-	2
9		CO3	BTL-	2
10		CO3	BTL-	2
PART – B (5 x 16 = 80 Marks)				
11. (a)		CO1	BTL-	
(Or)				
11. (b)		CO1	BTL-	
12.(a)		CO1	BTL-	
(Or)				
12.(b)		CO1	BTL-	
13.(a)		CO2	BTL-	
(Or)				

13.(b)		CO2	BTL-	
14.(a)		CO2	BTL-	
(Or)				
14.(b)		CO2	BTL-	
15.(a)		CO3	BTL-	
(Or)				
15.(b)		CO3	BTL-	

BTL-1 – Remember, BTL-2 – Understand, BTL-3 – Apply, BTL- 4–Analyze, BTL-5 – Evaluate, BTL-6 – Create

COURSE COORDINATOR

EVALUATOR

HOD

S.A. ENGINEERING COLLEGE (An Autonomous Institution, Affiliated to Anna University) Accredited by NBA & NAAC with 'A' Grade ISO 9001:2015 Certified Institution		Register No:		
INTERNAL ASSESSMENT- II EVEN SEMESTER, 2024- 2025 DEPARTMENT OF _____ (Common to _____) SUB CODE-SUB NAME _____ SEMESTER- _____				
Time: 180 mins COURSE OBJECTIVES: • • • COURSE OUTCOMES: SUB CODE.3 : SUB CODE.4 : SUB CODE.5 :		MAXIMUM MARKS: 100		
		CO.No.	BTL	Marks
Answer ALL the Questions PART – A (10 x 2 = 20 Marks)				
1		CO3	BTL-	2
2		CO3	BTL-	2
3		CO4	BTL-	2
4		CO4	BTL-	2
5		CO4	BTL-	2
6		CO4	BTL-	2
7		CO5	BTL-	2
8		CO5	BTL-	2
9		CO5	BTL-	2
10		CO5	BTL-	2
PART – B (5 x 16 = 80 Marks)				
11. (a)		CO3	BTL-	
(Or)				
11. (b)		CO3	BTL-	
12.(a)		CO4	BTL-	
(Or)				
12.(b)		CO4	BTL-	
13.(a)		CO4	BTL-	
(Or)				

13.(b)		CO4	BTL-	
14.(a)		CO5	BTL-	
(Or)				
14.(b)		CO5	BTL-	
15.(a)		CO5	BTL-	
(Or)				
15.(b)		CO5	BTL-	

BTL-1 – Remember, BTL-2 – Understand, BTL-3 – Apply, BTL- 4–Analyze, BTL-5 – Evaluate, BTL-6 – Create

COURSE COORDINATOR

EVALUATOR

HOD



Examination Reform Policy

November 2018

ALL INDIA COUNCIL FOR TECHNICAL EDUCATION
New Delhi, Mumbai, Varanasi, Kharagpur, New Bhopal, Chennai

ASSESSMENT STRATEGY FOR OUTCOME-BASED EDUCATION

1. Mapping Program Outcomes to Assessment (Examinations)

Graduate attributes (GAs) articulate the generic abilities to be looked for in a graduate of any undergraduate degree program. They form the Program Outcomes (POs) that reflect the skills, knowledge and abilities of graduates regardless of the field of study. This does not mean that POs are necessarily independent of disciplinary knowledge –rather, these qualities may be developed in various disciplinary contexts.

In outcome-based education, a “design down” process is employed which moves from POs to Course Outcomes (COs) and outcomes for individual learning experiences. Outcomes at each successive level need to be aligned with, and contribute to, the program outcomes.

Courses are the building blocks of a program. Teaching strategies, learning activities, assessments and resources should all be designed and organized to help students achieve the learning outcomes at the course level. In the assessment activities, students demonstrate their level of achievement of the course learning outcomes. In a constructively aligned program, the courses are carefully coordinated to ensure steady development or scaffolding from the introduction to mastery of the learning outcomes, leading to achievement of the intended POs. For the effectiveness of the program, the achievement of POs is crucial which needs to be proven through accurate and reliable assessments.

2. Two-step Process for Bringing Clarity to POs

POs give useful guidance at the program level for the curriculum design, delivery and assessment of student learning. However, they represent fairly high-level generic goals that are not directly measurable. Real observability and measurability of the POs at course level is very difficult. To connect high-level learning outcomes (POs) with course content, course outcomes and assessment, there is a necessity to bring further clarity and specificity to the program outcomes [5]. This can be achieved through the following two-step process of identifying Competencies and Performance Indicators (PI).

- (1) Identify Competencies to be attained: For each PO define competencies –different abilities implied by program outcome statement that would generally require different assessment measures. This helps us to create a shared understanding of the competencies we want students to achieve. They serve as an intermediate step to the creation of measurable indicators.

Example:

Program Outcome (Attribute 3)

Design:

PO3: Design/Development of Solutions: Design solutions for complex engineering problems and

design system components or processes that meet the specified needs with appropriate consideration for public health and safety, and cultural, societal, and environmental considerations.

Competencies

1. Demonstrate an ability to define a complex, open-ended problem in engineering terms.
 2. Demonstrate an ability to generate a diverse set of alternative design solutions.
 3. Demonstrate an ability to select the optimal design scheme for further development.
 4. Demonstrate an ability to advance an engineering design to the defined end state.
- (2) Define Performance Indicators: For each of the competencies identified, define performance Indicators (PIs) that are explicit statements of expectations of the student learning. They can act as measuring tools in assessment to understand the extent of attainment of outcomes. They can also be designed to determine the appropriate achievement level or competency of each indicator so that instructors can target and students can achieve the acceptable level of proficiency.

Example:

For the Competency -2

Demonstrate an ability to generate a diverse set of alternative design solutions

Performance Indicators:

1. Apply formal idea generation tools to develop multiple engineering design solutions
2. Build models, prototypes, algorithms to develop a diverse set of design solutions
3. Identify the functional and non-functional criteria for evaluation of alternate design solutions.

It should be noted that, when we consider the program outcome, it looks like, it can be achieved only in the Capstone project. But if we consider the competencies and performance indicators, we start seeing the opportunities of addressing them (and hence PO) in various courses of the program.

Once the above process is completed for the program, the assessment of COs for all the courses is designed by connecting assessment questions (used in various assessment tools) to the PIs. By following this process, where examination questions map with PIs, we get clarity and better resolution for the assessment of COs and POs. The pictorial representation of the process is given in Fig. 1

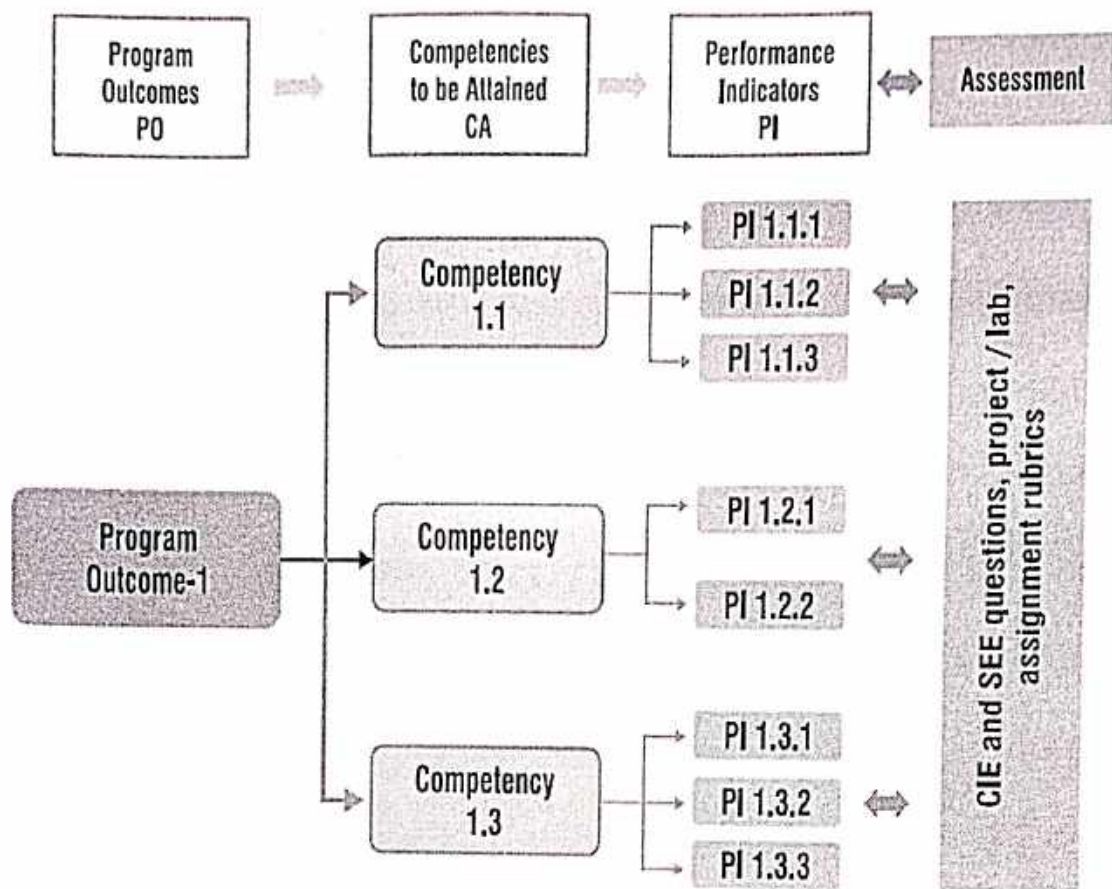


Fig. 1: Connecting POs to Assessment

PO 11: Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

Competency	Indicators
11.1 Demonstrate an ability to evaluate the economic and financial performance of an engineering activity	11.1.1 Describe various economic and financial costs/benefits of an engineering activity 11.1.2 Analyze different forms of financial statements to evaluate the financial status of an engineering project
11.2 Demonstrate an ability to compare and contrast the costs/benefits of alternate proposals for an engineering activity	11.2.1 Analyze and select the most appropriate proposal based on economic and financial considerations.
11.3 Demonstrate an ability to plan/manage an engineering activity within time and budget constraints	11.3.1 Identify the tasks required to complete an engineering activity, and the resources required to complete the tasks. 11.3.2 Use project management tools to schedule an engineering project, so it is completed on time and on budget.

PO 12: Life-long learning: Recognise the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Competency	Indicators
12.1 Demonstrate an ability to identify gaps in knowledge and a strategy to close these gaps	12.1.1 Describe the rationale for the requirement for continuing professional development 12.1.2 Identify deficiencies or gaps in knowledge and demonstrate an ability to source information to close this gap
12.2 Demonstrate an ability to identify changing trends in engineering knowledge and practice	12.2.1 Identify historic points of technological advance in engineering that required practitioners to seek education in order to stay current 12.2.2 Recognize the need and be able to clearly explain why it is vitally important to keep current regarding new developments in your field
12.3 Demonstrate an ability to identify and access sources for new information	12.3.1 Source and comprehend technical literature and other credible sources of information 12.3.2 Analyze sourced technical and popular information for feasibility, viability, sustainability, etc.

The above table can be used for most of the engineering programs. However, for Computer Science & Engineering/ Information Technology programs it requires some modifications.

A suggestive list of competencies and associated performance indicators for Computer Science & Engineering/ Information Technology Programs is given in Appendix- A.

IMPROVING STRUCTURE AND QUALITY OF ASSESSMENTS

For improving the structure and quality of assessment in various engineering programs following points need to be remembered:

1. In Indian engineering education system, written examinations play a major role in assessing the learning and awarding of grades to the student. Universities and colleges give highest weightage to the outcomes of the written examinations in overall grading. Questions raised in the examination/test papers play an important role in defining the level of learning the student is expected to achieve in the courses and hence in the program. Since assessment drives learning, the design of question papers needs to go beyond the mere test of memory recall. They also need to test higher-order abilities and skills.
2. Written examinations assess a very limited range of outcomes and cognitive levels. Particularly in the courses, where course outcomes (COs) cover a broad range of expectations, written examinations alone will not be sufficient to make valid judgements about student learning. A wide range of assessment methods (e.g., term papers, open-ended problem-solving assignments, course/lab project rubrics, portfolios etc.) need to be employed to ensure that assessment methods match with learning outcomes.
3. It is advisable to formulate assessment plans for each of the course in the program that brings clarity to the following:
 - a. Alignment of assessment with learning outcome of the course
 - b. Level of learning (cognitive) student is expected to achieve
 - c. Assessment method to be adapted

The method to align examination questions/assessment to COs and hence POs was discussed in the section-1. The following sections discuss the application of Bloom's taxonomy framework to create the optimal structure of examination papers to test the different cognitive skills.

1. Bloom's Taxonomy for Assessment Design

Bloom's Taxonomy provides an important framework to not only design curriculum and teaching methodologies but also to design appropriate examination questions belonging to various cognitive levels. Bloom's Taxonomy of Educational Objectives developed in 1956 by Benjamin Bloom [6] was widely accepted by educators for curriculum design and assessment. In 2001, Anderson and Krathwohl modified Bloom's taxonomy [7] to make it relevant to the present-day requirements. It attempts to divide learning into three types of domains (cognitive, affective, and behavioural) and then defines the level of performance for each domain. Conscious efforts to map the curriculum and assessment to these levels can help the programs to aim for higher-level abilities which go beyond remembering or understanding, and require application, analysis, evaluation or creation.

Revised Bloom's taxonomy in the cognitive domain includes thinking, knowledge, and application of knowledge. It is a popular framework in engineering education to structure the assessment as it characterizes complexity and higher-order abilities. It identifies six levels of competencies within the cognitive domain (Fig. 2) which are appropriate for the purposes of engineering educators.

According to revised Bloom's taxonomy, the levels in the cognitive domain are as follows:

Level	Descriptor	Level of attainment
1	Remembering	Recalling from the memory of the previously learned material
2	Understanding	Explaining ideas or concepts
3	Applying	Using the information in another familiar situation
4	Analysing	Breaking information into the part to explore understandings and relationships
5	Evaluating	Justifying a decision or course of action
6	Creating	Generating new ideas, products or new ways of viewing things

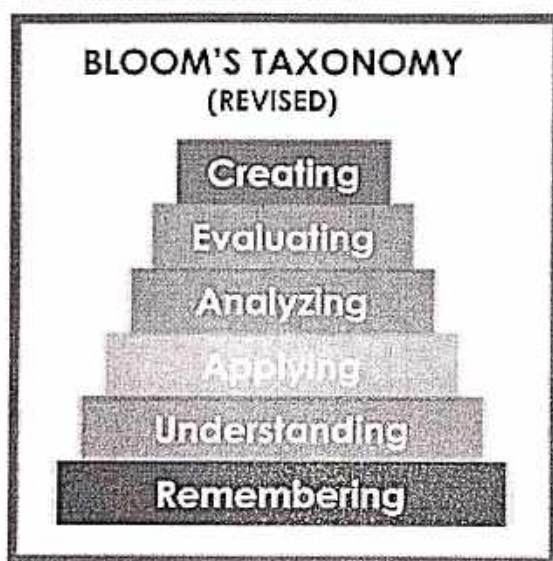


Fig. 2: Revised Bloom's Taxonomy

Bloom's taxonomy is hierarchical, meaning that learning at the higher level requires that skills at a lower level are attained.

2. Action Verbs for Assessment

Choice of action verbs in constructing assessment questions is important to consider. Quite often, the action verbs are indicators of the complexity (level) of the question. Over time, educators have come up with a taxonomy of measurable verbs corresponding to each of the Bloom's cognitive levels [8]. These verbs help us not only to describe and classify observable knowledge, skills and abilities but also to frame the examination or assignment questions that are appropriate to the level we are trying to assess.

Suggestive list of skills/ competencies to be demonstrated at each of the Bloom's level and corresponding cues/ verbs for the examination/ test questions is given below:

Level	Skill Demonstrated	Question cues / Verbs for tests
1. Remember	- Ability to recall of information like facts, conventions, definitions, jargon, technical terms, classifications, categories, and criteria - ability to recall methodology and procedures, abstractions, principles, and theories in the field - knowledge of dates, events, places - mastery of subject matter	list, define, tell, describe, recite, recall, identify, show, label, tabulate, quote, name, who, when, where
2. Understand	- understanding information - grasp meaning - translate knowledge into new context - interpret facts, compare, contrast - order, group, infer causes - predict consequences	describe, explain, paraphrase, restate, associate, contrast, summarize, differentiate interpret, discuss
3. Apply	- use information - use methods, concepts, laws, theories in new situations - solve problems using required skills or knowledge - Demonstrating correct usage of a method or procedure	calculate, predict, apply, solve, illustrate, use, demonstrate, determine, model, experiment, show, examine, modify
4. Analyse	- break down a complex problem into parts - Identify the relationships and interaction between the different parts of a complex problem - identify the missing information, sometimes the redundant information and the contradictory information, if any	classify, outline, break down, categorize, analyze, diagram, illustrate, infer, select
5. Evaluate	- compare and discriminate between ideas - assess value of theories, presentations - make choices based on reasoned argument - verify value of evidence - recognize subjectivity - use of definite criteria for judgments	assess, decide, choose, rank, grade, test, measure, defend, recommend, convince, select, judge, support, conclude, argue, justify, compare, summarize, evaluate
6. Create	- use old ideas to create new ones - Combine parts to make (new) whole, - generalize from given facts - relate knowledge from several areas - predict, draw conclusions	design, formulate, build, invent, create, compose, generate, derive, modify, develop, integrate

It may be noted that some of the verbs in the above table are associated with multiple Bloom's Taxonomy levels. These verbs are actions that could apply to different activities. We need to keep in mind that it's the skill, action or activity we need students to demonstrate that will determine the contextual meaning of the verb used in the assessment question.

3. Assessment Planning

While using Bloom's taxonomy framework in planning and designing of assessment of student learning, following points need to be considered:

1. Normally the first three learning levels; remembering, understanding and applying and to some extent fourth level analysing are assessed in the Continuous Internal Evaluation (CIE) and Semester End

APPENDIX

Sample questions for Bloom's Taxonomy Levels

Appendix-B

SAMPLES QUESTIONS FOR BLOOMS TAXONOMY LEVELS:

1. REMEMBER

Skill Demonstrated	Question Ques / Verbs for tests
- Ability to recall of information like, facts, conventions, definitions, jargon, technical terms, classifications, categories, and criteria - ability to recall methodology and procedures, abstractions, principles, and theories in the field - knowledge of dates, events, places - mastery of subject matter	list, define, describe, state, recite, recall, identify, show, label, tabulate, quote, name, who, when, where, etc.

Sample Questions:

1. State Ohm's law
2. List the physical and chemical properties of silicon
3. List the components of A/D converter
4. List the arithmetic operators available in C in increasing order of precedence.
5. Define the purpose of a constructor.
6. Define the terms: Sensible heat, Latent heat and Total heat of evaporation
7. List the assembler directives.
8. Describe the process of galvanisation and tinning
9. Write truth table and symbol of AND, OR, NOT, XNOR gates
10. Define the terms: Stress, Working stress and Factor of safety.
11. What is the difference between declaration and definition of a variable/function?
12. List the different storage class specifiers in C.
13. What is the use of local variables?
14. What is a pointer to a pointer?
15. What are the valid places for the keyword "break" to appear?
16. What is a self-referential structure?

2. UNDERSTAND

Skill Demonstrated	Question Ques / Verbs for tests
• understanding information • grasp meaning • translate knowledge into new context • interpret facts, compare, contrast • order, group, infer causes • predict consequences	describe, explain, paraphrase, restate, associate, contrast, summarize, differentiate interpret, discuss

Sample Questions:

1. Explain the importance of sustainability in Engineering design
2. Explain the behaviour of PN junction diode under different bias conditions
3. Describe the characteristics of SCR and transistor equivalent for a SCR
4. Explain the terms: Particle, Rigid body and Deformable body giving two examples for each.
5. How many values of the variable num must be used to completely test all branches of the following code fragment?

```

if (num > 0)
    if (value < 25)
    {
        value = 10 * num;
        if (num < 12)
            value = value / 10;
    }
else
    Value = 20 * num;
else
    Value = 30 * num
  
```

6. Discuss the effect of Make in India initiative on the Indian manufacturing Industry.
7. Summarise the importance of ethical code of conduct for engineering professionals
8. Explain the syntax for 'for loop'.
9. What is the difference between including the header file with-in angular braces < > and double quotes " "?
10. What is the meaning of base address of the array?
11. What is the difference between actual and formal parameters?
12. Explain the different ways of passing parameters to the functions.
13. Explain the use of comma operator (,).
14. Differentiate between entry and exit controlled loops.
15. How is an array different from linked list?

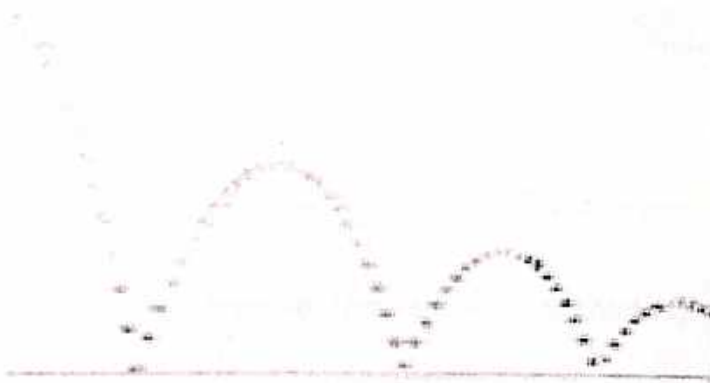
3. APPLY

Skill Demonstrated	Question Ques / Verbs for tests
- use information - use methods, concepts, laws, theories in new situations - solve problems using required skills or knowledge - Demonstrating correct usage of a method or procedure	calculate, predict, apply, solve, illustrate, use, demonstrate, determine, model, experiment, show, examine, modify

Sample Questions:

- Model and realize the following behaviors using diodes with minimum number of digital inputs.
 - Turning on of a burglar alarm only during night time when the locker door is opened.
 - Providing access to an account if either date of birth or registered mobile number or both are correct.
 - Updating the parking slot empty light in the basement of a shopping mall.
- One of the resource persons needs to address a huge crowd (nearly 400 members) in the auditorium. A system is to be designed in such a way that everybody attending the session should be able to hear properly and clearly without any disturbance. Identify the suitable circuit to boost the voice signal and explain its functionality in brief.
- A ladder 5.0 m long rests on a horizontal ground & leans against a smooth vertical wall at an angle 20° with the vertical. The weight of the ladder is 900 N and acts at its middle. The ladder is at the point of sliding, when a man weighing 750 N stands on a rung 1.5 m from the bottom of the ladder. Calculate the coefficient of friction between the ladder & the floor.
- A ball is dropped from 6 meters above a flat surface. Each time the ball hits the surface after falling a distance h , it rebounds a distance rh . What will be the total distance the ball travels in each of the following cases.

(a) $r > 1$ (b) $0 < r < 1$ (c) $r = 1$



- The region bounded by the curves $y = e^{(-1)x}$, $y = 0$, $x = 1$, and $x = 5$ is rotated about the x -axis. Use Simpson's Rule with $n = 8$ to estimate the volume of the resulting solid.
- An electric train is powered by machine which takes the supply from 220 V DC rail running above the train throughout. Machine draws current of 100 A from the DC rail to account for high torque during starting and runs at 700 r.p.m initially. Calculate the new speed of the train once it picks up the speed

where the torque output required is only 70% of starting torque. Assume the motor has a resistance of 0.1Ω across its terminals.

7. Write an algorithm to implement a stack using queue.
8. A single array $A[1..MAXSIZE]$ is used to implement two stacks. The two stacks grow from opposite ends of the array. Variables $top1$ and $top2$ ($top1 < top2$) point to the location of the topmost element in each of the stacks. What is the condition for "stack full", if the space is to be used efficiently.
9. Consider the following table of arrival time and burst time for three processes P0, P1 and P2.

Process	Arrival time	Burst Time
P0	0 ms	9 ms
P1	1 ms	4 ms
P2	2 ms	9 ms

The pre-emptive shortest job first scheduling algorithm is used. Scheduling is carried out only at arrival or completion of processes. What is the average waiting time for the three processes?

10. A CPU generates 32-bit virtual addresses. The page size is 4 KB. The processor has a translation look-aside buffer (TLB) which can hold a total of 128-page table entries and is 4-way set associative. What is the minimum size of the TLB tag?

4. ANALYZE

Skill Demonstrated	Question Ques / Verbs for tests
- break down a complex problem into parts. - Identify the relationships and interaction between the different parts of complex problem	classify, outline, break down, categorize, analyse, diagram, illustrate, infer, select

Sample Questions:

1. A class of 10 students consists of 5 males and 5 females. We intend to train a model based on their past scores to predict the future score. The average score of females is 60 whereas that of male is 80. The overall average of the class is 70. Give two ways of predicting the score and analyse them for fitting model.
2. Suppose that we want to select between two prediction models, M1 and M2. We have performed 10 rounds of 10-fold cross-validation on each model, whereas the same data partitioning in round one is used for both M1 and M2. The error rates obtained for M1 are 30.5, 32.2, 20.7, 20.6, 31.0, 41.0, 27.7, 26.0, 21.5, 26.0. The error rates for M2 are 22.4, 14.5, 22.4, 19.6, 20.7, 20.4, 22.1, 19.4, 16.2, 35.0. Comment on whether one model is significantly better than the other considering a significance level of 1%.
3. Return statement can only be used to return a single value. Can multiple values be returned from a function? Justify your answer.
4. Bob wrote a program using functions to find sum of two numbers whereas Alex wrote the statements to find the sum of two numbers in the `main()` function only. Which of the two methods is efficient in execution and why?
5. Carly wants to store the details of students studying in 1st year and later on wishes to retrieve the

information about the students who score the highest marks in each subject. Specify the scenario where the data can be organized as a single 2-D array or as multiple 1-D arrays.

6. Dave is working on a Campus Management Software but is unable to identify the maximum number of students per course. He decided to implement the same using arrays but discovered that there is memory wastage due to over-provisioning. Which method of memory storage should be used by Dave and how it can be implemented using C?
7. Albert is working on a 32-bit machine whereas Julie is working on a 64-bit machine. Both wrote the same code to find factorial of a number but Albert is unable to find factorial of a number till 9 whereas Julie is able to find the factorial of higher number. Identify the possible reason why Albert is unable to find the factorial. Suggest some changes in the code so that Albert can handle bigger inputs.
8. While writing a C code, the problem faced by the programmers is to find if the parenthesis is balanced or not. Write an algorithm to check if the parenthesis in C code are balanced. Initially your code should work for balanced { and } braces.
9. Swapping of the data in a linked list can be performed by swapping the contents in the linked list. Can the contents of a linked list be swapped without actually swapping the data?

5. EVALUATE

Skill Demonstrated	Question Ques / Verbs for tests
- compare and discriminate between ideas - assess value of theories, presentations - make choices based on reasoned argument - verify value of evidence - recognize subjectivity - use of definite criteria for judgments	assess, decide, choose, rank, grade, test, measure, defend, recommend, convince, select, judge, support, conclude, argue, justify, compare, summarize, evaluate

6. CREATE

Skill Demonstrated	Question Ques / Verbs for tests
- use old ideas to create new ones - Combine parts to make (new) whole, - generalize from given facts - relate knowledge from several areas - predict, draw conclusions	design, formulate, build, invent, create, compose, generate, derive, modify, develop, integrate

Both higher order cognitive skills 'Evaluate' and 'Create' are difficult to assess in time-limited examinations. These need to be assessed in variety of student works like projects, open ended problem-solving exercises etc. Typical examples of problem statements or need statements which need higher order abilities to solve are given below

Sample Problem / Need statements:

1. Automatic tethering of milking machine to the udder of a cow. A milk dairy wants to automate the milking process. The milking process involves attaching the milking cups to the teats. Design a system for the same.
2. An electric vehicle uses LiON batteries. The batteries have to be charged and get discharged during use.

The batteries require continuous monitoring during charging and discharging so that they remain healthy and yield a long life. Design a system to monitor and manage the health of the batteries.

3. A Biotech industry needs automation for filling its product into 20 ltr bottles. Design a system to meter the flow into the bottles so that each bottle has 20 ltr of the liquid. There will be more than one filling station and the system has to monitor all the filling stations as well as keep count of the total production on a daily basis.
4. Microwave Doppler radar with a range of 9m are available for motion detection. Design a surround view monitoring system for a 3 wheeler to detect human obstacles while the vehicle is in motion.
5. Design a system to assist the driver by using cameras to detect lane markers and pedestrians while the vehicle is in motion.
6. Develop a small size USB 2.0 / 3.0 CMOS camera system which can be used for industrial inspection, medical applications, microscopy, etc. The system should be able to capture the image quickly and be able to process the captured image and then store it also

S.A. ENGINEERING COLLEGE

(Autonomous-Institute Level Research Centre-Affiliated to
Anna University)

Accredited by NBA & NAAC with 'A' Grade
ISO 9001:2015 Certified Institution

Register No:

INTERNAL ASSESSMENT-I
ODD SEMESTER, 2025- 2026
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
(Common to B.Tech IT & B.Tech CSBS)
CS1501A - OPERATING SYSTEMS

Time: 180 mins

SEMESTER- V

MAXIMUM MARKS: 100

COURSE OBJECTIVES:

- To understand the basic concepts, functions, processes and threads of operating systems.
- To analyse scheduling algorithms and understand the concept of deadlocks.
- To analyse various memory management schemes.
- To understand I/O management and file systems.
- To be familiar with the basics of Linux system and Mobile OS like iOS and Android.

COURSE OUTCOMES:

- CS1501A.1: Understand the basic concepts, functions, processes and threads of operating systems
- CS1501A.2: Analyze various scheduling algorithms, Understand deadlock, prevention and avoidance algorithms.
- CS1501A.3: Compare and contrast various memory management schemes.

CO. No. BTL Marks

Answer ALL the Questions
PART - A (10 x 2 = 20 Marks)

1	What is the purpose of system calls?	CO1	BTL-1	2
2	Does timesharing differ from multiprogramming? If so, How?	CO1	BTL-2	2
3	What is the basic principle behind the working of a virtual machine?	CO1	BTL-1	2
4	List the differences between a process and a program.	CO1	BTL-1	2
5	What is context switching?	CO2	BTL-1	2
6	Define starvation and aging in CPU scheduling.	CO2	BTL-1	2
7	State the difference between mutex and binary semaphore.	CO2	BTL-1	2
8	What can the operating system do to recover from deadlock?	CO2	BTL-1	2
9	Differentiate internal fragmentation from external fragmentation.	CO3	BTL-2	2
10	What is the purpose of paging the page tables?	CO3	BTL-1	2

PART - B (5 x 13 = 65 Marks)

11.(a)	Demonstrate your understanding of virtualization by outlining the historical evolution of virtual machines. Also, apply relevant methods and techniques used to integrate virtualization into operating systems.	CO1	BTL-3	13
--------	--	-----	-------	----

- (Or)
- 11.(b) Classify and illustrate the diverse categories of system calls offered by a standard operating system, providing a specific example for each. CO1 BTL-3 13
- 12.(a) Explain different operating system structures with a neat sketch. CO1 BTL-2 13
- (Or)
- 12.(b) Describe the dual-mode and multimode operations of operating systems, providing a detailed examination of each mode. CO1 BTL-2 13
- 13.(a) Discuss the concept of interacting processes and explore two distinct methodologies for implementing interaction between processes. CO2 BTL-3 13
- (Or)
- 13.(b) Assume the following processes arrive for execution at the time indicated and also mention with the length of the CPU burst time given in milliseconds. CO2 BTL-3 13

Job	Burst Time	Priority	Arrival Time
A	10	5	0
B	6	2	0
C	7	4	1
D	4	1	1
E	5	3	2

(i) Give a Gantt chart illustrating the execution of these processes using FCFS, Round Robin (quantum=5) and Priority (Preemptive and Non Preemptive).

(ii) Calculate the average waiting time and average turnaround time for each of the above scheduling algorithm.

- 14.(a) Compare and Contrast Preemptive and Non-preemptive CPU scheduling algorithms with examples. Discuss which type is better suited for interactive systems. CO2 BTL-4 13
- (Or)
- 14.(b) Critically examine the Reader-Writer Problem and propose a solution that prevents writer starvation. Write a pseudo-code or algorithm to support your explanation. CO2 BTL-4 13
- 15.(a) Given memory partitions of 100K, 500K, 200K, 300K and 600K (in order). How would each of the First-fit, Best-fit and Worst-fit algorithms place processes of 412K, 317K, 112K and 326K (in order)? Which algorithm makes the most efficient use of memory? CO3 BTL-3 13
- (Or)
- 15.(b) Discuss the following memory management techniques with diagrams. CO3 BTL-3 13
- (i) Partition Allocation Methods. (07)
- (ii) Paging and Translation Look-aside Buffer. (06)

PART - C (1x 15 = 15 Marks)

- 16.(a) Analyze the concept of deadlock and examine various strategies used for deadlock recovery in operating systems. Break down and CO2 BTL-4 15

evaluate the methods involved in detecting deadlocks.

(Or)

- 16.(b) A system has 4 processes (P0 to P3) and 3 resource types (A, B, C). CO2 BTL-4 15
The total instances of each resource are A=3, B=17, C=16, D=12.
Current allocation and maximum requirement are:

Process	Allocation	Max
	A B C D	A B C D
P0	0 1 1 0	0 2 1 0
P1	1 2 3 1	1 6 5 2
P2	1 3 6 5	2 3 6 6
P3	0 6 3 2	0 6 5 2
P4	0 0 1 4	0 6 5 6

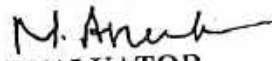
(i) Find the need matrix.

(ii) Is the system in a safe state? Show the safe sequence if it exists.

BTL-1 – Remember, BTL-2 – Understand, BTL-3 – Apply, BTL- 4–Analyze, BTL-5 – Evaluate, BTL-6 – Create



COURSE COORDINATOR


EVALUATOR


HOD

MM F04 G

S.A. ENGINEERING COLLEGE, CHENNAI-77
 (An Autonomous Institution, Affiliated to Anna University)
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
QUESTION PAPER EVALUATION FORM
ACADEMIC YEAR: 2024-2025 (ODD)

ACADEMIC YEAR: 2024-2025 (ODD)						
Year: III		Subject Code: CS1501A				
Sem: V		Subject Name: OPERATING SYSTEMS				
INTERNAL ASSESSMENT- II (SET-1)						
Question No	BTL1	BTL2	BTL3	BTL4	BTL5	BTL6
PART A						
1.	✓					
2.		✓				
3.	✓					
4.	✓					
5.	✓					
6.	✓					
7.	✓					
8.	✓					
9.		✓				
10.	✓					
PART B						
11. (a)			✓			
11. (b)						
12. (a)		✓				
12. (b)						
13. (a)			✓			
13. (b)						
14. (a)				✓		
14. (b)			✓			
15. (a)						
15. (b)						
PART C						
16. (a)				✓		
16. (b)						
% of Mapping	16	17	39	28		

Based on Bloom's Taxonomy

BTL 1: REMEMBER

Exhibit memory of previously learned material by recalling facts, terms, basic concepts and answers.

BTL 2: UNDERSTAND

Demonstrate understanding of facts and ideas by organizing, comparing, translating, interpreting, giving descriptions and stating main ideas.

BTL 3: APPLY

Solve problems to new situation by applying acquired knowledge, facts, techniques and rules in a different way.

BTL 4: ANALYZE

Examine and break information into parts by identifying motives or causes. Make inferences and find evidence to support generalizations.

BTL 5: EVALUATE

Present and defend opinions by making judgment about information, validity of ideas, or quality of work based on a set of criteria.

BTL 6: CREATE

Compile information together in a different way by combining elements in a new pattern or proposing alternative solutions.

Remarks:


FACULTY INCHARGE


PROGRAMME COORDINATOR


HOD / CSE

Reg. No. :

Question Paper Code : 2310330

B.E DEGREE END SEMESTER EXAMINATIONS, NOV / DEC 2023.

Fourth Semester

Department of Computer Science and Engineering

CS1403 / CS1403A - OBJECT ORIENTED ANALYSIS AND DESIGN

(Regulations 2020 / 2020A)

TIME : 180 Minutes

Maximum : 100 Marks

Answer ALL Questions.

PART A – (10 x 2 = 20 marks)

- | | | | |
|-----|--|------|-----|
| 1. | Define Object Oriented analysis and Design. | BTL1 | CO1 |
| 2. | State the importance of include and extend and relationship. | BTL2 | CO1 |
| 3. | Define a domain model. | BTL1 | CO2 |
| 4. | Why domain model is called a visual dictionary? | BTL1 | CO2 |
| 5. | State a scenario where Activity diagram is used. | BTL1 | CO3 |
| 6. | Define System sequence diagram. | BTL1 | CO3 |
| 7. | What is GOF? | BTL1 | CO4 |
| 8. | Define pattern 'Controller'. | BTL2 | CO4 |
| 9. | Write the issues in OO testing. | BTL2 | CO5 |
| 10. | What is GUI testing? | BTL1 | CO5 |

PART B – (5 x 13 = 65 marks)

11. a) Briefly explain the different phases of Unified Process with a neat diagram. (13) BTL2 CO1

Or

- b) What does Use Case Diagram represent? Give an example. (13) BTL2 CO1

- | | | | | |
|-------|--|------|------|-----|
| 12.a) | Explain Aggregation, Composition and Association with suitable examples. | (13) | BTL2 | CO2 |
|-------|--|------|------|-----|

Or

- | | | | | |
|--------|--|------|-------|-----|
| b) | Describe in detail about how Conceptual class hierarchies are found. | (13) | BT1.2 | CO2 |
| 13. a) | Summarize in detail using an example how methods can be created from interaction diagrams. | (13) | BTL2 | CO3 |

Or

- b) i) Write short notes on Exception and error handling. (6) BTL2 CO3
ii) What are the basic elements of deployment diagrams? Explain. (7) BTL2 CO3
14. a) Write short notes on i) Attribute visibility ii) Parameter Visibility iii) Local visibility iv) Global visibility. (13) BTL1 CO4

Or

- b) Discuss in detail about 3 types of Design Patterns and their types with Example. (13) BTL1 CO4
15. a) What is OMT? Explain with examples (13) BTL4 CO5
i. Object Model
ii. OMT Dynamic Model.

Or

- b) Compare unit testing and Integration testing in OO Testing. (13) BTL4 CO5

PART C – (1 x 15 = 15 marks)

16. a) Draw the Use case diagram, Activity diagram and Interaction diagram for the following specification: (15) BTL2 CO4
A Coffee Vending Machine dispenses coffee to customers. Customers order coffee by selecting a recipe from a set of recipes. Customers pay for the coffee using coins. Change is given back, if any, to the customers. The 'Service Staff' loads ingredients (coffee, powder, milk, sugar, water, chocolate) into the coffee machine. The 'Service Staff' can also add a recipe by indicating the name of the coffee, the units of coffee powder, milk, sugar, water and chocolate to be added as well as the cost of the coffee.

Or

- b) A University conducts examinations and the results are announced. Prepare a report for the following; Print the marks in the register number order semester wise for each department. (15) BTL2 CO4
• Print the Arrear list semester wise.
• Prepare a Rank list for each department.
• Prepare the final aggregate mark list for final year students.
Identify the problem statement and Design and Explain the classes for each sequence. Design the Use case, Class diagrams for designing this system.

S.A. ENGINEERING COLLEGE, CHENNAI-600 077

(Autonomous-Institute Level Research Centre-Affiliated to Anna University)

THEORY EXAMINATION – APRIL/MAY 2025

Theory Subject Feedback Form

Class/Sem : II A, II B, II C / IV & III IT / VI
 Date/Session of the Exam : 06-05-2025 / FN
 Question Paper Code :
 Subject code/Name : CS1403A / Object Oriented Analysis and Design
 Handled by :
 1. Mr. M Balasubramanian (II A)
 2. Mrs. L Dhanani (II B)
 3. Mrs. S Gnanapriya (II C)
 4. Mrs. A Seetha (III IT)

COMMENTS:

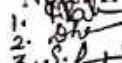
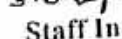
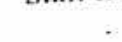
S.NO	QUESTION WAS DIFFICULT	MISTAKES/OMMISSION OF VALUES	OUT OF SYLLABUS
		NIL	

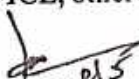
Dept/Year	Section	Pass Percentage in Assessment Tests		Expected Pass Percentage in End Sem Exam
		IA-I	IA-II	
CSE / II	A	77.05	85.25	Above 90
CSE / II	B	77.05	86.89	
CSE / II IT / III	C	60.70 80	60.70 93.1	

Q.P. Matching Ratio between Internal Assessment and End Semester Theory Examination: 83%

Whether it has to be updated to COE Web Portal: No

Note: If YES, submit to COE OFFICE, otherwise submit to Dept.

1. 
 2. 
 3. 
 Staff Incharge


 HOD


 CHIEF SUPERINTENDENT

Reg. No. 111923CS01044

Question Paper Code : 2500304

B.E / B.Tech DEGREE END SEMESTER EXAMINATIONS, APR / MAY 2025

Fourth / Sixth Semester

Department of Computer Science and Engineering

CS1403 / CS1403A - OBJECT ORIENTED ANALYSIS AND DESIGN

(Common to Computer Science and Engineering / Information Technology)

(Regulations 2020 / 2020A)

Maximum : 100 Marks

TIME : 180 Minutes

Answer ALL Questions.

PART A - (10 x 2 = 20 marks)

- | | | | |
|------------------|--|------|-----|
| TA-01 1. (3M Q) | Define Use case. | BTL1 | CO1 |
| TA-01 2. (11M Q) | Compare Include and Extend use case relationships. | BTL2 | CO1 |
| TA-01 3. (6M Q) | Rank the 3 strategies to find conceptual class. | BTL1 | CO2 |
| TA-01 4. (14M Q) | Mention the guidelines to create domain model. | BTL1 | CO2 |
| | Organize and Show the relationship between Interaction and Class diagram with example. | BTL2 | CO3 |
| TA-02 6. (11M Q) | Define Event, State and Transition. | BTL1 | CO3 |
| TA-02 7. (9M Q) | State the benefits of Low coupling. | BTL1 | CO4 |
| TA-02 8. (14M Q) | Interpret the need of Information Expert. | BTL2 | CO4 |
| TA-02 9. [15M Q] | Illustrate the impact object orientation in testing. | BTL2 | CO5 |
| TA-02 10. (8M Q) | Define test plan? What are its components? | BTL1 | CO5 |

PART B - (5 x 13 = 65 marks)

- | | | | | |
|----------------------|---|------|------|-----|
| TA-01 11 a) (11M Q) | Describe in detail the various phases of Rational Unified Process. | (13) | BTL2 | CO1 |
| | Or | | | |
| TA-01 11 b) (11M Q) | Outline the use case modelling process with suitable examples. | (13) | BTL2 | CO1 |
| TA-01 12 a) (12M Q) | Identify the class diagram for Airline Reservation System. Find and draw the conceptual classes for the same. | (13) | BTL2 | CO2 |
| | Or | | | |
| b) | Explain the relationship between sequence diagrams and Use case with an example. | (13) | BTL2 | CO2 |
| TA-01 13. a) (11M Q) | Illustrate the Communication diagram with an example. | (8) | BTL3 | CO3 |
| ii) | Explain the basic elements of Deployment diagram. | (5) | BTL3 | CO3 |

Or

A hospital management system that consists of components like patient registration, appointment scheduling, doctor and staff management, billing, and medical records. The system needs to interact with external services like insurance companies and pharmacies. Design a UML Component Diagram for the hospital management system, showing the key components and how they interact with external services. (13) BTL3 CO3

14. a) Explain GRASP design patterns in object modeling with examples. (13) BTL2 CO4

Or

i) Differentiate Adapter and Bridge pattern. (7) BTL2 CO4

ii) Explain Mapping design to code process with examples. (6) BTL2 CO4

15. a) Illustrate the process of developing Test Cases and Test Plans with relevant example. (13) BTL3 CO5

Or

b) Explain Rumbaugh's Object Modelling Technique for model for Banking application. (13) BTL3 CO5

4 x 13 = 52 Marks

PART C - (1 x 15 = 15 marks)

16. a) A large-scale enterprise resource planning (ERP) system for a multinational company. The system includes modules for finance, human resources, sales, procurement, and logistics, with a large number of developers working on different parts of the system. At what stage in the software development process would you recommend using UML Package Diagrams? Justify your answer. (15) BTL4 CO4

Or

b) For Airline Ticket reservation system explain and draw the following UML diagrams (15) BTL4 CO4

i) Sequence diagram (booking a ticket) (7)

ii) Activity diagram. (4)

iii) State chart diagram. (4)

1 x 15 = 15 Marks

S.A. ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University)
Accredited by NBA & NAAC with 'A' Grade
ISO 9001:2015 Certified Institution

INTERNAL ASSESSMENT-I EVEN SEMESTER, 2024-2025 B.E COMPUTER SCIENCE AND ENGINEERING (Common to B.Tech. IT) CS1403A-OBJECT ORIENTED ANALYSIS AND DESIGN

Time: 180 mins

SEMESTER-IV (CSE)/ VI (IT)

MAXIMUM MARKS: 100

COURSE OBJECTIVES:

- To understand the fundamentals of object analysis and design.
- To understand and differentiate Unified Process from other approaches.
- To design with static UML diagrams and identify the concept.
- To design with the UML dynamic and implementation diagrams.
- To design the software with appropriate design patterns.
- To test the software against its requirements specification

COURSE OUTCOMES:

CS1403A.1: Express software design with UML diagrams

CS1403A.2: Design software applications using OO concepts

CS1403A.3: Identify various scenarios based on software requirements

CO.No. BT Mark

Answer ALL the Questions PART - A (10 x 2 = 20 Marks)

1	What is Object Oriented Analysis and Design?	CO1	BTL-1	2
2	Mention the benefits of Unified Process.	CO1	BTL-1	2
3	What is Use-case Modeling?	CO1	BTL-1	2
4	What are pre conditions and post conditions?	CO1	BTL-1	2
5	What is the relationship of a conceptual super class to a subclass?	CO2	BTL-2	2
6	Define a) Conceptual Class b) Specification Class c) Implementation Class?	CO2	BTL-1	2
7	What are derived attributes? Give an example.?	CO2	BTL-2	2
8	When to use class diagram?	CO2	BTL-1	2
9	Enumerate the structure of the class diagram for ATM system	CO3	BTL-2	2
10	Define Communication Diagram.	CO3	BTL-1	2

PART - B (5 x 13 = 65 Marks)

11(a)	Apply the knowledge of the Unified Process phases by briefly explaining each with a diagram, showcasing the interconnected stages for a comprehensive understanding.	CO1	BTL-3	13
-------	--	-----	-------	----

(Or)

11(b)	Illustrate the necessity of relationships in a use case diagram and with suitable example show the various relationships and how it helps in designing the use case diagram.	CO1	BTL-3	13
12(a)	Design Use case and Class diagram for an ATM application and explain the purpose of each diagram.	CO1	BTL-3	13

(Or)

- 12.(b) Apply and design the Use case diagram for the following specification:
A Coffee Vending Machine dispenses coffee to customers. Customers order coffee by selecting a recipe from a set of recipes. Customers pay for the coffee using coins. Change is given back, if any, to the customers. The 'Service Staff' loads ingredients (coffee, powder, milk, sugar, water, chocolate) into the coffee machine. The 'Service Staff' can also add a recipe by indicating the name of the coffee, the units of coffee powder, milk, sugar, water and chocolate to be added as well as the cost of the coffee.

CO1 BTL-3

- 13.(a) Enumerate briefly about how to find Conceptual class hierarchies by outlining the steps involved in their identification and development

CO2 BTL-3

(Or)

- 13.(b) Apply the understanding of Aggregation, Composition and Association by providing relevant examples that demonstrate the distinctions between these concepts in the context of object-oriented modeling and class relationships.

CO2 BTL-3

- 14.(a) Illustrate how the iterative process of refining a domain model enhances its representation of a problem domain by explaining the concept and providing a relevant example.

CO2 BTL-3

(Or)

- 14.(b) Apply the class diagram problem statement and design a class diagram for a "Library Management System". State the functional requirements you are considering

CO2 BTL-3

- 15.(a) Illustrate with a real time example of how to draw a UML component diagram and explain the components in detail.

CO3 BTL-2

(Or)

- 15.(b) Examine the need for various UML diagrams and draw the sequence and collaboration diagrams for a house keeping system in a five star hotel.

CO3 BTL-2

PART - C (1 x 15 = 15 Marks)

- 16.(a) A University conducts examinations and the results are announced. Prepare a report for the following; Print the marks in the register number order semester wise for each department.

- Print the Arrear list semester wise.
- Prepare a Rank list for each department.
- Prepare the final aggregate mark list for final year students.

CO1 BTL-3

Analyze the problem statement and Design the Class diagrams for this system.

(Or)

- 16.(b) Apply the problem statement and design a class diagram for a "Library Management System". State the functional requirements you are considering.

CO1 BTL-3

BTL-1 - Remember, BTL-2 - Understand, BTL-3 - Apply, BTL-4 - Analyze, BTL-5 - Evaluate, BTL-6 - Create


COURSE COORDINATOR


EVALUATOR,


HOD

S.A. ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University)
Accredited by NBA & NAAC with 'A' Grade
ISO 9001:2015 Certified Institution

Register No: AB-03602,
115, 134
Pre-58

INTERNAL ASSESSMENT-II
EVEN SEMESTER, 2024-2025

Common To

B.E COMPUTER SCIENCE AND ENGINEERING AND B.Tech INFORMATION TECHNOLOGY
CS1403A-OBJECT ORIENTED ANALYSIS AND DESIGN

Time: 180 mins

SEMESTER-VI

MAXIMUM MARKS: 100

COURSE OBJECTIVES:

- To understand the fundamentals of object analysis and design.
- To understand and differentiate Unified Process from other approaches.
- To design with static UML diagrams and identify the concept.
- To design with the UML dynamic and implementation diagrams.
- To design the software with appropriate design patterns.
- To test the software against its requirements specification

COURSE OUTCOMES:

- CS1403A.3: Identify various scenarios based on software requirements
CS1403A.4: Transform UML based software design into pattern based design using design patterns
CS1403A.5: Understand the various testing methodologies for OO software
CS1403A.6 Design and develop a case study on object oriented projects

CO.No. BTL Marks

Answer ALL the Questions
PART - A (10 x 2 = 20 Marks)

1	List out the reasons for using dynamic UML Diagrams	CO3	BTL-1	2
2	What are the relationship Sequence and Communication Diagrams?	CO3	BTL-1	2
3	State the reason for modeling a package diagram.	CO4	BTL-2	2
4	"Coupling should be low" - Justify	CO4	BTL-1	2
5	Define high cohesion	CO4	BTL-1	2
6	What are the various types of coupling?	CO4	BTL-1	2
7	Define software quality assurance	CO5	BTL-1	2
8	List the steps needed to create a test plan.	CO5	BTL-1	2
9	Write about GUI testing.	CO5	BTL-1	2
10	Define Test cases	CO5	BTL-1	2

PART - B (5 x 13 = 65 Marks)

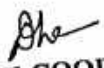
11. (a) Outline the steps in modeling and explain the activity diagram with an example. Why to use an activity diagram and state its merits and demerits? CO3 BTL-3 13
- (Or)
11. (b) Illustrate the state chart diagram and its components and draw the state chart diagram for ATM system. CO3 BTL-3 13

12(a)	Examine the need for various UML diagrams and draw the sequence and collaboration diagrams for a house keeping system in a five star hotel	CO3	BTL-3	13
	(Or)			
12(b)	Outline the steps in modeling and explain the activity diagram with an example. Why to use an activity diagram and state its merits and demerits.	CO3	BTL-3	13
13(a)	With an illustrated example, brief on adapter pattern.	CO4	BTL-3	13
	(Or)			
13(b)	Examine the mapping design to code and elaborate the steps for converting a NextGen POS system design into code.	CO4	BTL-3	13
14(a)	Explain GRASP technique and explain the nine object oriented design principles.	CO4	BTL-3	13
	(Or)			
14(b)	Examine the application of GoF design patterns with suitable examples. What are the importance of using GoF patterns?	CO4	BTL-3	13
15(a)	Define testing and explain the four types of testing strategies.	CO5	BTL-2	13
	(Or)			
15(b)	Discuss briefly about Issues in OO Testing.	CO5	BTL-2	13

PART - C (1 x 15 = 15 Marks)

16(a)	Examine the various concepts encompassed within Software Quality Assurance (SQA) also provide a comprehensive analysis detailing the fundamental principles, methodologies, and techniques involved in ensuring software quality, with real-world examples	CO5	BTL-3	15
	(Or)			
16(b)	Apply the methodologies and approaches involved in conducting class testing and GUI testing also provide a comprehensive analysis, including the key considerations, tools, and techniques essential for ensuring the robustness and usability of object-oriented class structures and graphical user interfaces.	CO5	BTL-3	15

BTL-1 – Remember, BTL-2 – Understand, BTL-3 – Apply, BTL-4 – Analyze,
BTL-5 – Evaluate, BTL-6 – Create


COURSE COORDINATOR


EVALUATOR


MOD



S.A. ENGINEERING COLLEGE

An Autonomous Institution, Affiliated to Anna University, Chennai
Accredited by NBA, NAAC A Grade and ISO 9001:2015 Certified Institution

Academic Year : 2024-2025 ODD
Year : IYR

Batch : 2023-2027
Semester : III-SEMESTER

Programme : CSE
Max Mark : 100

Section : A

INTERNAL ASSESSMENT 1 - SCORE SHEET

S.No.	Reg No.	Student Name	CS1301A			IT1301A(CS			EC1306A(CS			MA1303A(CS			CS1302A(CS			PASS/FAIL	TOTAL MARK
			DS			OOPS			DPSD			DM			SE				
			21-08-2024			22-08-2024			23-08-2024			27-08-2024			29-08-2024				
			60	40	100	60	40	100	60	40	100	60	40	100	60	40	100		
24	111923CS01082	MADHAN KUMAR M	40	40	80	32	40	72	11	39	50	41	40	81	41	40	81	FAIL	364
25	111923CS01084	MAGESHWARI M	48	40	88	35	40	75	32	40	72	54	40	94	44	40	84	PASS	413
26	111923CS01085	MAGESWARAN P	42	40	82	42	40	82	42	40	82	56	40	96	51	40	91	PASS	433
27	111923CS01087	MANIKANDAN-A	34	40	74	35	40	75	20	39	59	52	40	92	32	40	72	FAIL	372
28	111923CS01088	MANISHAA SRI S	54	40	94	55	40	95	42	40	82	55	40	95	49	40	89	PASS	455
29	111923CS01089	MANOJ KUMAR M	25	40	65	00	40	40	2	38	40	13	40	53	14	40	54	FAIL	252
30	111923CS01093	MONISHA S	54	40	94	57	40	97	54	40	94	48	40	88	49	40	89	PASS	462
31	111923CS01097	MUKUNDAN J	25	40	65	7	40	47	9	39	48	16	40	56	13	40	53	FAIL	269
32	111923CS01098	MURALI KRISHNA	39	40	79	32	40	72	22	40	62	48	40	88	40	40	80	FAIL	381
33	111923CS01099	NANDHINI S	43	40	83	30	40	70	38	40	78	32	40	72	48	40	88	PASS	391
34	111923CS01105	PAVAN KUMAAR M	42	40	82	52	40	92	36	40	76	41	40	81	37	40	77	PASS	408
35	111923CS01118	PRIYANKA V	39	40	79	44	40	84	56	40	96	58	40	98	52	40	92	PASS	449
36	111923CS01122	RASHIKA R	52	40	92	45	40	85	43	40	83	47	40	87	35	40	75	PASS	422
37	111923CS01125	RINAY KISHORE K	43	40	83	AB	40	40	AB	40	40	34	40	74	34	40	74	FAIL	311
38	111923CS01128	ROHITH K	33	40	73	34	40	74	12	39	51	39	40	79	35	40	75	FAIL	352
39	111923CS01129	ROSELIN ANGELO	31	40	71	36	40	76	32	40	72	34	40	74	44	40	84	PASS	377



An Autonomous Institution, Affiliated to Anna University, Chennai
Accredited by NBA, NAAC A Grade and ISO 9001:2015 Certified Institution

Section : A

INTERNAL ASSESSMENT 1 - SCORE SHEET

S.No.	Reg No.	Student Name	CS1301A		IT1301A(CS		EC1306A(CS		MA1303A(CS		CS1302A(CS		PASS/FAIL	TOTAL MARK				
			DS	OOPS	DPSD	DM	SE											
								21-08-2024	22-08-2024	23-08-2024	27-08-2024	29-08-2024						
			60	40	100	60	40	100	60	40	100	60	40	100				
1	111923CS01004	ABDUL KALAM Y	54	40	94	38	40	78	AB	38	38	23	40	63	46	40	86	359
2	111923CS01009	ADITYA S	25	40	65	5	40	45	5	39	44	44	40	84	19	40	59	297
3	111923CS01012	AKRAM KHAN S	35	40	75	37	40	77	37	40	77	34	40	74	34	40	74	377
4	111923CS01017	ARIPUTHIRAN A	24	40	64	10	40	50	3	38	41	17	40	57	19	40	59	271
5	111923CS01018	ASHVITHA S	56	40	96	48	40	88	50	40	90	58	40	98	55	40	95	467
6	111923CS01020	BALA MUKESH M	34	40	74	34	40	74	8	38	46	37	40	77	23	40	63	334
7	111923CS01021	BALAGANESH S	18	40	58	6	40	46	00	38	38	2	40	42	00	40	40	224
8	111923CS01024	BHARATH S	42	40	82	30	40	70	40	40	80	55	40	95	49	40	89	416
9	111923CS01029	DEEPIKA C	49	40	89	53	40	93	53	40	93	60	40	100	56	40	96	471
10	111923CS01034	DHANUSRID	24	40	64	30	40	70	20	40	60	24	40	64	35	40	75	333
11	111923CS01040	DIVYABARATHI J	57	40	97	58	40	98	54	40	94	60	40	100	54	40	94	483
12	111923CS01043	ENIYAN M E	24	40	64	10	40	50	30	40	70	20	40	60	18	40	58	302
13	111923CS01044	GANAPATHIN	41	40	81	35	40	75	8	38	46	45	40	85	19	40	59	346
14	111923CS01045	GOKULRAV E	31	40	71	5	40	45	3	40	43	18	40	58	30	40	70	287
15	111923CS01047	HARINID	40	40	80	38	40	78	38	40	78	49	40	89	43	40	83	408
16	111923CS01050	HARISH V	30	40	70	19	40	59	11	39	50	17	40	57	8	40	48	244

Course Code	DS	OOPS	DPSD	DM	SE
No.of Students Present	61	59	58	61	61
No.of Students Absent	0	2	3	0	0
No.of Students Pass	48	42	33	45	47
No.of Students Fail	13	17	25	16	14
Pass Percentage	78.69	68.85	54.10	73.77	77.05
Subject Average	40.45	34.57	29.84	40.64	37.50

Overall Pass Percentage = $32 / 61 = 52.46\%$

Signature of Class Advisor

HOD



INTERNAL ASSESSMENT 1 - SCORE SHEET

Academic Year : 2024-2025 ODD			Batch : 2023-2027		Programme : CSE		Section : A									
Year : IIYR			Semester : III-SEMESTER		Max Mark : 100											
S.No.	Reg No.	Student Name	CS1301A		IT1301A(CS		EC1306A(CS		MA1303A(CS		CS1302A(CS		PASS/FAIL	TOTAL MARK		
			DS		OOPS		DPSD		DM		SE					
			21-08-2024		22-08-2024		23-08-2024		27-08-2024		29-08-2024					
			60	40	100	60	40	100	60	40	100	60			40	100
47	111923CS01149	SHARVANI G	43	40	83	49	40	89	46	40	86	56	40	96	PASS	440
48	111923CS01152	SHRIRAM S DAS	46	40	86	AB	40	40	AB	40	40	57	40	97	FAIL	351
49	111923CS01154	SHUSMI R	55	40	95	49	40	89	38	40	78	56	40	96	PASS	444
50	111923CS01160	SREERAG K K	30	40	70	17	40	57	4	39	43	14	40	54	FAIL	283
51	111923CS01161	SRINIVAS S P	21	40	61	8	40	48	10	40	50	13	40	53	FAIL	270
52	111923CS01163	SWATHIKA N	48	40	88	49	40	89	55	40	95	59	40	99	PASS	460
53	111923CS01164	SWETHA S	57	40	97	53	40	93	58	40	98	58	40	98	PASS	477
54	111923CS01165	SWETHA T	54	40	94	37	40	77	34	40	74	42	40	82	PASS	418
55	111923CS01167	THANUJA J	48	40	88	51	40	91	49	40	89	49	40	89	PASS	447
56	111923CS01171	VAISALI S	56	40	96	58	40	98	58	40	98	56	40	96	PASS	483
57	111923CS01174	VENNALA SAMEERA	55	40	95	51	40	91	45	40	85	49	40	89	PASS	455
58	111923CS01176	VIGNESH R	21	40	61	6	40	46	7	38	45	15	40	55	FAIL	279
59	111923CS01177	VISHNU DHARSHAN	49	40	89	48	40	88	42	40	82	56	40	96	PASS	441
60	2316182L	ARAVIND S	28	40	68	20	40	60	17	38	55	18	40	58	FAIL	314
61	2316186L	THIRUSANKAN B	25	40	65	25	40	65	7	38	45	14	40	54	FAIL	289



S. A. ENGINEERING COLLEGE, CHENNAI-77

An Autonomous institution Affiliated to Anna University, Chennai.
Accredited by NBA, NAAC with "A" Grade and ISO 9001:2015 certified institution
Poonamallee - Avadi Main Road, Thiruverkadu, Chennai - 600 077.

OFFICE OF THE CONTROLLER OF EXAMINATIONS REPORT OF THE SCRUTINY COMMITTEE

End Semester Theory Examinations Nov / Dec 2025

Sub Code /Sub Name: _____ Department : _____

Semester/Year : _____ Date : _____

S.No.	MEMBER	AU CODE/ FACULTY ID	AFFILIATION
1.			
2.			

1.	The question paper(s) have been set strictly in accordance with the syllabus and regulation.	Yes/No
2.	The question paper contains the particulars like Regulations, Degree, Branch, Semester, Subject Code, Title, Duration, Maximum marks.	Yes/No
3.	Marks allotted to each questions are clearly mentioned.	Yes/No
4.	Part B and Part C have a maximum of three sub divisions.	Yes/No
5.	Number system in the question paper template is followed.	Yes/No
6.	No repetition of Questions.	Yes/No
7.	No abbreviations.	Yes/No
8.	Representation and weightage have been given to all the units as per the question paper template.	Yes/No
9.	No missing data or incorrect data for the given questions, (If the data is irrelevant, the scrutiny team can provide relevant data)	Yes/No
10.	The drawings, photographs and pictorial data in the question paper are legible.	Yes/No
11.	Student who attends all the requisite choices shall complete within the stipulated time.	Yes/No
12.	The questions are according to the level of the programme.	Yes/No
13.	The questions are not ambiguous, or/and out of syllabus.	Yes/No
14.	The questions are from Text Books and Reference Books provided in the syllabus.	Yes/No
15.	Special Instructions regarding Codes, Data Books, Charts and Tables, Drawing & Graph sheets to be supplied or to be permitted are mentioned. (if applicable)	Yes/No
16.	Bloom's Taxonomy Action Verbs are in prescribed format and both questions in the OR choice have same BTL and covers same CO	Yes/No

Comments on Question Paper:

Sl. No	Question No	Comments

Quality of Question Paper:

☐

Low

☐

Moderate

☐

High

Declaration:-

I / We hereby declare that the question paper has been scrutinized with full knowledge and syllabus. We assure that the confidentiality of the question paper will be maintained with utmost care.

MEMBER

CHAIRMAN

NAME & SIGNATURE

NAME & SIGNATURE

2. Process of Assignment Preparation and Evaluation

2. Process of Assignment Preparation and Evaluation

Assignments are an integral component of the Continuous Internal Assessment (IA) system. They are designed to assess students' understanding, analytical ability, problem-solving, and application of knowledge beyond classroom instruction.

Each Course Handling Faculty or Course Coordinator prepares the assignment questions in alignment with:

- Course Outcomes (COs)
- Program Outcomes (POs) / Program Specific Outcomes (PSOs)
- Bloom's Taxonomy Levels (BTL)

Assignments are structured to evaluate higher-order cognitive skills—Apply, Analyse, Evaluate, and Create through activities such as design tasks, programming exercises, mini-projects, problem-solving.

A. Assignment Preparation Process

Step	Activity	Responsibility	Outcome / Record
1	Prepare assignment questions mapped with COs and BTL	Course Faculty	CO-based Assignment Sheet
2	Announce deadlines, evaluation rubrics, and unit coverage	Course Faculty	Assignment Schedule
3	Evaluate submissions using standard rubrics	Course Faculty	Evaluated Assignment Sheets
4	Record and enter assignment marks separately in ERP / internal mark register (out of 50) for each IA cycle	Course Faculty	Assignment Evaluation Register
5	Review attainment and suggest improvements	Department Committee	CO Attainment Report

- For every semester, two internal assessment (IA) exams are conducted, each carrying 100 marks.
- In addition, assignment marks (50 marks each) are entered separately for both IA-I and IA-II.
- Each course consists of five units, and assignment evaluation ensures 20 marks are allocated per unit ($20 \times 5 = 100$ total assignment marks across the semester).
- This structure provides equal weightage to all units and ensures continuous learning and assessment throughout the course duration.

B. Example – Course: Object-Oriented Programming

Assignments in Object-Oriented Programming (OOP) emphasize the application of core programming concepts encapsulation, inheritance, polymorphism, and multithreading to solve real-time computational problems.

Each assignment is mapped with relevant COs and evaluated using standard rubrics to maintain objectivity and uniformity.

Example Assignment:

Students are required to design a synchronized Java program demonstrating ordered thread execution. This reinforces knowledge of multithreading, concurrency control, synchronization mechanisms, and exception handling, aligning with higher-order BTL levels.

Sample Rubric for Object-Oriented Programming Assignment

Criteria	Weightage (%)	Description
Understanding of Exception & Synchronization Concepts	25	Demonstrates clarity in thread handling, synchronization, and exception management
Problem Identification and Analysis	20	Defines problem requirements and formulates logical solution steps
Code Implementation, Syntax & Quality	25	Uses Java APIs (synchronized, wait/notify, locks, etc.) efficiently and correctly
Correctness of Output Logic	20	Produces accurate output with correct execution flow
Presentation and Documentation	10	Provides well-documented, formatted, and readable code

Total Marks: 20

Mapped CO: CO3

Related POs / PSOs: PO 1, 2, 3, 4, 8, 9; PSO 1, 2

Bloom's Levels: Apply (BTL 3), Analyse (BTL 4)

Assignment Outcome

- Students demonstrated the ability to design concurrent programs with correct synchronization and thread coordination using Java APIs.
- Improved conceptual understanding of exception handling, synchronization, and inter-thread communication.
- Encouraged independent learning through resources from Infosys Springboard and NPTEL.
- Ensured unit-wise assessment consistency by linking each assignment (20 marks per unit) with overall IA evaluation.

C. Integration of Digital Assignments

To promote independent and industry-oriented learning, students are encouraged to complete digital assignments through online platforms offering practical exercises and certification courses.

S.No	Platform	Course Area / Focus
1	MATLAB	Simulation, FEM, Project Design
2	Infosys Springboard	Technology & Career Skills
3	Coursera	Algorithmic & Technical Courses

4	AICTE PARAKH	Industry-Aligned Courses
5	NPTEL / SWAYAM	Core & Advanced Engineering
6	NASSCOM	Deep Tech & Digital Skills
7	Wipro TalentNext	Project-Based Learning
8	CISCO Networking Academy	Networking & IoT
9	AWS Academy	Cloud Computing

These platforms support self-paced learning, skill development, and exposure to global academic standards, enhancing the continuous internal evaluation framework.

D. Quality Assurance in Assignment Evaluation

Evaluation Parameter	Quality Measure	Action Taken
Question Alignment	Verified CO-PO-BTL Mapping	Approved by Course Coordinator
Evaluation Consistency	Rubric-based Evaluation	Monitored by Department Committee
Assignment Marks Entry	50 marks entered separately for each IA (unit-wise, 20 marks per unit)	Verified by Course Coordinator & HoD
Attainment Review	Based on performance analysis	Incorporated into next course cycle
Faculty Feedback	Reviewed in Department Meeting	Assignment questions and rubrics refined annually



SUBJECT: IT3301/OBJECT ORIENTED PROGRAMMING
CLASS/SEM: II CSE /III
ASSIGNMENT II

Marks 20

Regno: 111924CS01157
 Name : S. Mohit Kumar
 Year & Section: II - D
 Date of submission: 22-9-25

INNOVATIVE ASSIGNMENT 3

Part A (20 marks)

INNOVATIVE ASSIGNMENT		CO	PO
Part A (20 marks)		3	1,2,3,4,5
S.No	Question		
1.	Problem Statement A bank wants to conduct examinations for recruitment. You need to develop an application for the applicants to submit their details by implementing the classes based on the class diagram and description given below. ⊗ Applicant ▪ name: String ▪ jobProfile: String ▪ age: int ⊙ getName(): String ⊙ setName(name: String): void ⊙ getJobProfile(): String ⊙ setJobProfile(jobProfile: String): void ⊙ getAge(): int ⊙ setAge(age: int): void ⊗ Validator ⊙ validateName(name: String): boolean ⊙ validateJobProfile(jobProfile: String): boolean ⊙ validateAge(age: int): boolean ⊙ validate(applicant: Applicant): void ⊗ InvalidAgeException ⊙ InvalidAgeException(message: String) ⊗ InvalidJobProfileException ⊙ InvalidJobProfileException(message: String) ⊗ InvalidNameException ⊙ InvalidNameException(message: String)		
Note: The Applicant class is implemented and provided to you.			

Validator

Method Description

validateName(String name)

- Validate that the name is not null or empty. If the name is null or empty, return false, else return true.

validateJobProfile(String jobProfile)

- Validate that the jobProfile is either 'Associate' or 'Clerk' or 'Executive' or 'Officer'. If the jobProfile is valid, return true, else return false. Perform case insensitive comparison.

validateAge(int age)

- Validate that the age is between 18 and 30 (both inclusive). If the age is valid, return true, else return false.

validate(Applicant applicant)

- Validate the details of the applicant by calling the appropriate methods. If validation fails, throw user defined exceptions based on the below description.

Field violated	User defined exception	Exception message
name	InvalidNameException	Invalid name
jobProfile	InvalidJobProfileException	Invalid job profile
age	InvalidAgeException	Invalid age

Implement the required user defined exception classes.

Test the functionalities using the main method of the provided Tester class based on the below description.

- Create an object of Applicant class and set the values of all the instance variables.
- Validate the details of the applicant by invoking the validate() method of the Validator class.
- If all the details are valid, display 'Application submitted successfully!', else display appropriate error message.

Sample Input and Output										
Input										
<table><tr><th>Instance Variables</th><th>Values</th></tr><tr><td>name</td><td>Jenny</td></tr><tr><td>jobProfile</td><td>Clerk</td></tr><tr><td>age</td><td>25</td></tr></table>			Instance Variables	Values	name	Jenny	jobProfile	Clerk	age	25
Instance Variables	Values									
name	Jenny									
jobProfile	Clerk									
age	25									
Output										
Application submitted successfully!										
Input										
<table><tr><th>Instance Variables</th><th>Values</th></tr><tr><td>name</td><td>null</td></tr><tr><td>jobProfile</td><td>clerk</td></tr><tr><td>age</td><td>27</td></tr></table>			Instance Variables	Values	name	null	jobProfile	clerk	age	27
Instance Variables	Values									
name	null									
jobProfile	clerk									
age	27									
Output										
Invalid name										
Input										
<table><tr><th>Instance Variables</th><th>Values</th></tr><tr><td>name</td><td>Jenny</td></tr><tr><td>jobProfile</td><td>Assistant</td></tr><tr><td>age</td><td>25</td></tr></table>			Instance Variables	Values	name	Jenny	jobProfile	Assistant	age	25
Instance Variables	Values									
name	Jenny									
jobProfile	Assistant									
age	25									
Output										
Invalid job profile										
Note: Questions from Infosys springboard										
2	Scenario: Two threads are supposed to print two different sets of numbers (say, multiples of 5 and multiples of 100). The requirement is that first all multiples of 5 are printed in ascending order, and only after that, the second thread prints its multiples of 100. Threads may otherwise interleave unpredictably. Task: Write/design a solution using Java multithreading APIs (synchronized, wait/notify, locks, etc.) so that the output always respects the required order. Note: Questions from Nptel	3								
		1,2,3,4,5								

Rubrics

Part A	Excellent(4-5)	Good(3)	Fair(2)	Poor(1)
Understanding of Exception & Synchronization Mechanism Concept	4			
Identification of Problem in Case Study	4			
Code Implementation, Syntax & Code Quality		3		
Correctness of Output Logic		3		

Marks (out of 10): $14/20 = 7/10$

Faculty signature with date :

Shel
24/9/25

description:-

Exception handling in java is a mechanism to handle runtime errors so that the normal flow of the program is maintained.

It uses try, catch, finally, throw and throws keyword.

In this example.

We are creating a custom exception called Invalid

Invalidage Exception that is thrown when the user enters an age less than 18. This demonstrates custom exception creation, exception throwing and handling with multiple catch blocks.

Syntax:-

```
try {  
    // code that might throw an exception  
}
```

```
catch (Exception type Ex 1) {  
    // Handle exception type 1  
}
```

```
catch (Exception type 2: Ex 2) {  
    // Handle exception type 2  
}
```

```
finally {  
    // code that always executes  
}
```

```
throw new Exception Type ("message"); // to throw exception manually  
Public void method Name () throws Exception Type {  
    // method that might throw exception  
}
```

Concept used :

- custom exception class
- throw and throws
- try - catch - finally block
- Multiple catch handling
- Input validation with exception handling.
- Implementing the classes.

Code :-

```
import java.util.Scanner;

class InvalidAgeExceptionDemo {
    public InvalidAgeException (String message)
    {
        super (message);
    }
}

public class ExceptionDemo {
    public static void validateAge (int age) throws InvalidAgeException
    {
        if (age < 18)
        {
            throw new InvalidAgeException ("Age must be 18 or Above to Register :");
        }
        else
        {
            System.out.println ("Registration successfully :");
        }
    }

    public static void main (String [] args)
    {
        Scanner sc = new Scanner (System.in);
        try {
            System.out.println ("Enter your age :");
        }
    }
}
```

```

int age = sc.nextInt();
validate age (age);
}
catch (Invalid Age Exception e)
{
    System.out.println (" customed Exception caught : "+ e Invalid age ());
}
catch (Exception e)
{
    System.out.println ("Process completed . closing resources...");
    System.out.println (" General Exception : "+ e);
}
finally
{
    System.out.println ("Process completed . closing resources...");
    sc.close();
}
}
}

```

Explanation:

1. Custom Exception - Invalid age Exception is created by extending exception class
2. Throwing Exception - InvalidAge Age() method throw is used to raise the exception if the age is less than 18.
3. catching Exception - try block contains the risky code, catch handles the exception.
4. Multiple catch Blocks - First handles the custom Exception, second handles general exception.
5. Finally Block - Executes regardless of whether an exception is throw or not (used for resource cleanup)

Output:

Case 1:

Enter your age : 20

Registration Successfully

Process completed closing resources.....

Case 2:

Enter your age : 15

Custom Exception caught : Age must be 18 or above to register.

Process completed closing resources.....

Case 3 :-

Enter your age: abc

General Exception :

java.util.InputMismatchException

Process completed - closing resources.....

description

* Thread:

A thread is a lightweight unit of execution within a process.

Java supports threads using the thread class or implementing the runnable interface.

Syntax:-

```
class mythread extends Thread {  
    public void run() {  
        //code.  
    }  
}
```

* Synchronized

Ensures that only one thread can execute a block / method at a time
used to avoid race condition.

Syntax:-

```
synchronized (object) {  
    // body  
}  
  
(or)  
  
public synchronized void method name()  
{  
    // body  
}
```

* wait() and notify()

- wait() - causes the current thread to pause execution.
- notify() - wake up a single waiting thread.

- notifyAll() → wakes up all waiting thread.

Syntax:-

```
Synchronized (lock object) {
    while (condition - not met) {
        lock object.wait();
        // thread goes into waiting state
    }
    lock object.notify();
    // wake up waiting thread
}
```

Reentrant synchronization:-

Once a thread enters a synchronized block, it holds the intrinsic lock and can re-enter synchronized code prototype by the same lock.

Code:-

```
class NumberPrinter {
    Private boolean firstDone = false;
    Public synchronized void printMultiples of 5 (int limit) {
        for (int i = 5; i <= limit; i += 5) {
            System.out.println("5 -> " + i);
        }
        firstDone = true;
        notifyAll();
    }
    Public synchronized void printMultiples of 10 (int limit) {
        while (!firstDone) {
```

```

try {
    wait (0);
}
catch ( InterruptedException e ) {
    Thread.currentThread().interrupt();
}
}
for (int i = 100 ; i <= limit ; i += 100) {
    System.out.println("T2 -> " + i);
}
}
}
}

```

```

public class ThreadOrderExample {

```

```

    public static void main ( String [] args)
    {

```

```

        NumberPrinter printer = new NumberPrinter ();

```

```

        Thread t1 = new Thread () -> { printer.printMultipleOf 5 (50); };

```

```

        Thread t2 = new Thread () -> { printer.printMultipleOf 100 (500); };

```

```

        t2.start();

```

```

        t1.start();

```

```

    }

```

```

}

```

Sample output;

$T_1 \rightarrow 5$

$T_1 \rightarrow 10$

$T_1 \rightarrow 15$

$T_1 \rightarrow 20$

$T_1 \rightarrow 25$

$T_1 \rightarrow 30$

$T_1 \rightarrow 35$

$T_1 \rightarrow 40$

$T_1 \rightarrow 45$

$T_1 \rightarrow 50$

$T_2 \rightarrow 100$

$T_2 \rightarrow 200$

$T_2 \rightarrow 300$

$T_2 \rightarrow 400$

$T_2 \rightarrow 500$

description 2



SUBJECT: IT3301/OBJECT ORIENTED PROGRAMMING
CLASS/SEM: II CSE /III
ASSIGNMENT II

Marks 20

Regno: 111924CS01 256

Name: SREEMATHI . P

Year & Section: 11th & CSE-7

Date of submission: 6/08/25

ASSIGNMENT II

Part A (10 marks)

1. A) Write a simple Vehicle class that has fields for (at least) current speed, current direction in degrees, and owner name. Add a static field to your Vehicle class to hold the next vehicle identification number, and a non-static field to the Vehicle class to hold each car's ID number. Add two constructors to Vehicle: a no-arg constructor and one that takes an initial owner's name. Add a changeSpeed method that changes the current speed of the vehicle to a passed-in value and add a stop method that sets the speed to zero. Create an extended class called PassengerVehicle to add a capability for counting the number of seats available in the car and the number currently occupied. Provide a main method in PassengerVehicle to create a few of these objects and print them out.

(or)

- B) Write a simple Vehicle class that has fields for (at least) current speed, current direction in degrees, and owner name. Add a static field to your Vehicle class to hold the next vehicle identification number, and a non-static field to the Vehicle class to hold each car's ID number. Add two constructors to Vehicle: a no-arg constructor and one that takes an initial owner's name. Add a changeSpeed method that changes the current speed of the vehicle to a passed-in value and add a stop method that sets the speed to zero. Change Vehicle so that it has an EnergySource object reference, which is associated with the Vehicle in its constructor. EnergySource must be an abstract class, because a GasTank object's measure of fullness will differ from that of a Battery object. Put an abstract empty method in EnergySource and implement it in GasTank and Battery classes. Add a start method to Vehicle that ensures that the energy source isn't empty.

Part B (10 marks)

Across

2. Hiding internal details and showing functionality is

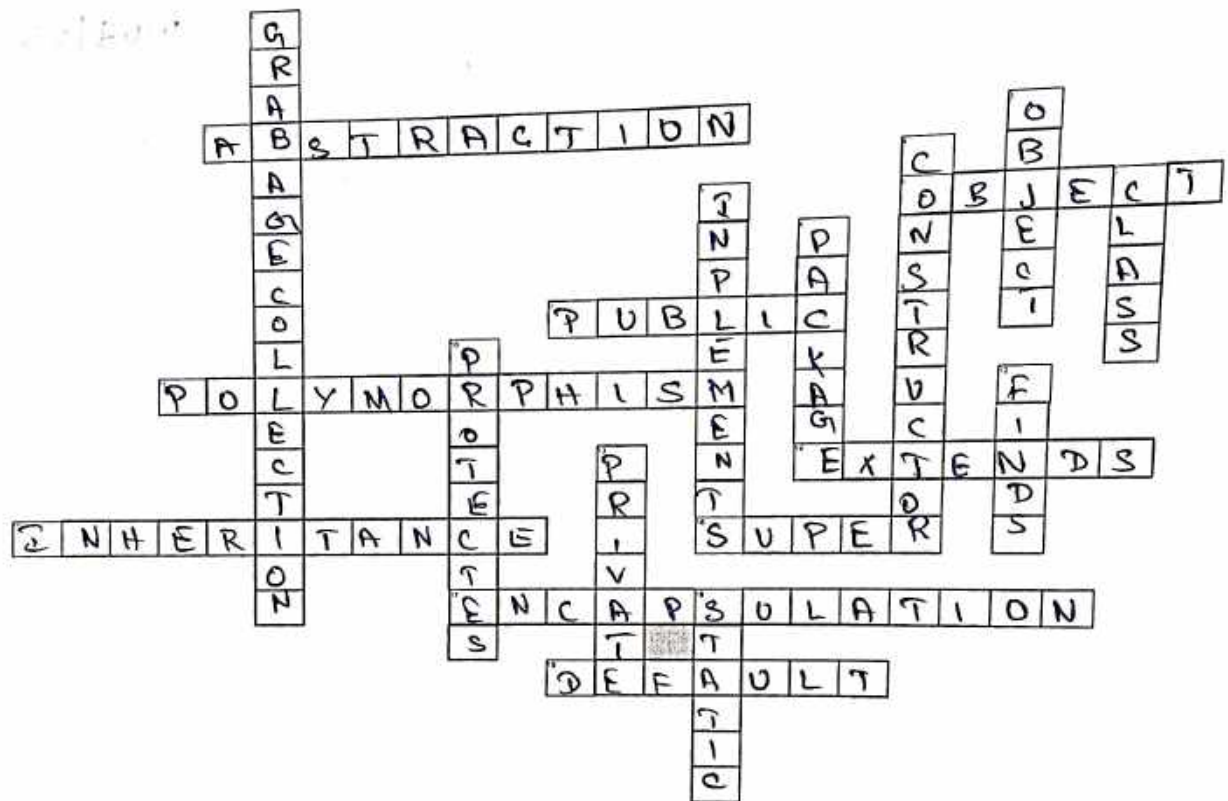
Across:

6. Any entity that has state and behaviour is known as
9. The is accessible everywhere
11. When one task is performed by different ways known as
14. keyword Used in a class declaration to specify the superclass; used in an interface declaration to specify one or more super interfaces.
15. When one object acquires all the properties and behaviours of parent object known as
16. can be used to refer immediate parent class instance variable or method.
17. Binding (or wrapping) code and data together into a single unit is known as
19. The is accessible only within package

Down

1. Java automatically frees the used memory for other uses
3. class which is the superclass of all classes
4. is a special type of method that is used to initialize the object.

5. keyword Included in a class declaration to specify one or more interfaces that are used by the current class.
7. Collection of objects is called. It is a logical entity.
8. is a group of similar types of classes, interfaces, and sub-packages.
10. the is accessible within a package and outside the package but through inheritance only
12. keyword is used to apply restrictions on class, method, and variable.
13. The is accessible only within class
18. Classes maintain one copy of class variables regardless of how many instances exist of that class.



at - A

Write a simple vehicle class that has fields for (at least) current speed, current direction in degrees, and owner name. Add a static field to your vehicle class to hold the next vehicle identification numbers, and a non-static field to the vehicle class to hold each car's ID number. Add two constructors to vehicle: a no-argument constructor and one that takes an initial owner's name. Add a changeSpeed method that changes the current speed of the vehicle to a passed-in value and add a stop method that sets the speed to zero. Create an extended class called passengerVehicle to add a capability for counting the number of seats available in the car and the number currently occupied. Provide a main method in passengerVehicle to create a few of these objects and print them out.

Sol

Description:

Vehicle class:

The vehicle class is the base class representing a general vehicle. It includes:

Fields (Attributes):

- * Double speed - The current speed of the vehicle.
- * Double direction - The current direction in degrees (0 to 360)
- * String ownerName - The name of the vehicle's owner
- * int vehicle ID - A unique ID for each vehicle
- * static int nextID - A static variable used to assign a

unique vehicle ID to each vehicle as its Created

Constructors :

* No - arguments Constructor - sets default values (OwnerName

"unknown", Speed = 0, direction = 0) and assign a unique ID.

* Parameterized Constructor - Takes an ownerName as input and assigns a unique ID.

Methods:

* ChangeSpeed (double newSpeed): updates the speed of the vehicle

* Stop(): Sets the speed to 0

* ToText(): Returns a string containing all details of the vehicle.

2) PassengerVehicle Class

PassengerVehicle is a subclass of vehicle, representing vehicles used to carry people.

⇒ Additional Fields:

* int totalSeats: Total number of seats in the vehicle.

* int seatsOccupied - Number of seats currently occupied.

⇒ Constructor:

* Accepts ownerName, totalSeats, and seatsOccupied. It calls the vehicle constructor to initialize the owner and other base fields.

Quizz

minor

Concept

Methods:

* getSeats Available () - Returns how many seats are still available (totalSeats - seatsOccupied).

* To text () - Adds seat information to the base class output using super.toText().

Main method:

The main() method does the following:

- 1) Creates 3 passenger vehicle objects with different owner names and seat data.
- 2) sets speed for two vehicles and stops one vehicle using changeSpeed () and stop ().
- 3) prints ^{out} the full details of all vehicles using toText ()

Code:

```
class vehicle {  
    double speed;  
    double direction;  
    String ownerName;  
    int vehicleID;  
  
    static int nextID = 1;  
    vehicle () {  
        ownerName = "Unknown";  
        speed = 0;  
        direction = 0;  
        vehicleID = nextID++;  
    }  
}
```

```
vehicle (String ownerName) {
```

```
    this.ownerName = "unknown" ownerName;
```

```
    Speed = 0;
```

```
    direction = 0;
```

```
    vehicle ID = nextID++;
```

```
}
```

```
void changeSpeed (double newSpeed) {
```

```
    speed = newSpeed;
```

```
}
```

```
void stop () {
```

```
    speed = 0;
```

```
}
```

```
String toString () {
```

```
    return "vehicle ID: " + vehicle ID + ", owner: " + ownerName + "
```

```
    speed: " + speed + " km/h, Direction: " + direction + ". " ;
```

```
}
```

```
}
```

```
class PassengerVehicle number extends vehicle {
```

```
    int totalSeats;
```

```
    int seatsOccupied;
```

```
PassengerVehicle (String ownerName, int totalSeats, int  
    seatsOccupied) {
```

```
    super (ownerName);
```

```
    this.totalSeats = totalSeats;
```

```
    this.seatsOccupied = seatsOccupied;
```

```
}
```

```
int getSeatsAvailable () {
```

```
    return totalSeats - seatsOccupied;
```

```
}
```

```
# getseats Available () {
```

```
String toText () {
```

```
    return super.toText () + ", Total seats : " + " , occupied :
```

```
    + seatsoccupied + " , Available : " + getseats Available ;  
}
```

```
Public Static void main (String args [])  
{
```

```
    PassengerVehicle Car 1 = new PassengerVehicle ("Alice", 5, 2);
```

```
    PassengerVehicle Car 2 = new PassengerVehicle ("Bob", 7, 5);
```

```
    PassengerVehicle Car 3 = new PassengerVehicle ("Charlie", 4, 4);
```

```
    Car 1 . changespeed (60);
```

```
    Car 2 . changespeed (80);
```

```
    Car 3 . Stop ();
```

```
    System . out . println ("passenger vehicle details : \n");
```

```
    System . out . println ("Car 1 . toText ()");
```

```
    System . out . println ("Car 2 . toText ()");
```

```
    System . out . println ("Car 3 . toText ()");
```

```
}  
}
```

Output :

Passenger Vehicle Details :

Vehicle ID : 1 , owner : Alice , Speed : 60.0 km/h , Total seats : 5 ,

occupied : 2 , Available : 3 ,

Vehicle ID : 2 , owner : Bob , Speed : 80.0 km/h , Total seats : 7 ,

occupied : 5 , Available : 2 .

Vehicle ID : 3, owner : Charlie, speed : 0.6 km/h, total

Seats : 4,

Occupied : 4, Available : 0



S.A. Engineering College, Chennai-77.
(Autonomous-Institute Level Research Centre-Affiliated to Anna University)
Department of Computer Science and Engineering
CS1501A-Operating Systems
Assignment-II Questions

AY/SEM: 2025-2026 (ODD)

Reg no:	111923CS01163
Name :	SWATHIKA N
Year & Section:	III & "A"
Date of submission:	1-9-25

S.NO	QUESTIONS
1.	Design and implement a functional program to simulate and compare multiple scheduling algorithms or synchronization mechanisms. Your program should collect experimental data on performance metrics such as throughput, fairness, and efficiency. Using this data, generate visual charts or graphs that effectively illustrate the comparative analysis. Submissions without actual program code and data-driven visualizations will not be accepted. Hint: To complete this assignment, first understand the key scheduling algorithms or synchronization mechanisms and design your program to simulate their process handling accurately. Implement code to model process states, execute the algorithms, and collect performance data such as throughput, fairness, and efficiency metrics. Use programming libraries to generate visual charts like Gantt or bar graphs that clearly compare the results. Test your program with diverse inputs, document your code well, and analyze the visuals to explain the differences in algorithm or mechanism performance.
2.	Use diagramming tools (UML, flowcharts) to visualize process interactions and deadlock cycles. Hint: To visualize process interactions and deadlock cycles, you can use UML interaction diagrams and flowcharts that capture dynamic behaviour and message flows among processes. For example, a sequence diagram shows the chronological order of messages exchanged between processes or threads, illustrating how synchronization and resource requests happen over time. Collaboration diagrams emphasize the structural organization of interacting processes. For deadlock cycles, resource allocation graphs or activity diagrams can be drawn to represent how processes request and hold resources, helping identify cycles that indicate deadlocks. Tools like Draw.io, Lucidchart, or Visual Paradigm make it easy to create these diagrams, clearly showing process states, communication, and synchronization points for better understanding and analysis.

ASSIGNMENT RUBRIC

Criteria	Excellent (20 pts)	Good (15 pts)	Fair (10 pt)	Poor (5 pt)
Program Functionality	Complete, correctly simulates multiple scheduling/synchronization algorithms; runs without errors	Simulates most algorithms correctly; minor bugs or missing edge cases	Partial implementation; some algorithms missing or incorrect outcomes	Program does not run or only implements algorithm incorrectly
Data Collection & Metrics	Accurately collects key performance data: throughput, fairness, efficiency; metrics well-defined	Collects data for most metrics with minor inaccuracies	Collects limited metrics or some data is unreliable	Metrics missing or incorrect; no meaningful data collected
Visualizations (Charts/Graphs)	Clear, well-labeled, and meaningful visual aids (bar charts, Gantt charts, etc.) effectively support comparative analysis	Visualizations present, mostly clear and relevant	Visual aids present but poorly labeled, incomplete, or only partly support analysis	Visualizations missing or irrelevant
Code Quality & Documentation	Code clean, modular, well-commented; functions/classes logically organized	Code mostly clean, some comments and basic organization	Code cluttered or repetitive; minimal comments	Code disorganized, hard to follow, lacks comments
Analysis & Interpretation	Includes insightful discussion of results; explains trade-offs between algorithms or mechanisms based on data/visuals	Provides basic explanation of results using data and charts	Limited or unclear interpretation of results	No meaningful analysis or explanation

OPERATING

3. Variations and

8.

OPERATING SYSTEM UNIT-II CROSSWORD PUZZLE

Across

3. Variable used to solve critical section problem and synchronization
8. Scheduling criterion that measures how busy the CPU is
10. Graph used in deadlock detection by showing processes waiting for others
12. Situation where processes wait indefinitely for resources
14. High-level synchronization construct that controls access to shared data
16. Time from submission until the first response
17. Problem involving multiple readers and writers on shared data

Down

1. Queue where processes wait for I/O completion
2. Scheduling criterion that measures number of processes completed per unit time
4. Method of ensuring deadlock cannot occur by resource allocation rules
5. Strategy using safe states to avoid deadlocks
6. Goal of CPU scheduling to increase number of completed processes
7. Act of saving and restoring process state during CPU switching
9. Scheduling where a running process can be interrupted

<https://crosswordlabs.com/view/os-unit-2>

1/2

8/20/25, 10:28 PM

OS UNIT-2 - Crossword Labs

19. Condition where a process holding resources waits for additional ones
20. Methods used after detecting deadlock, like process termination
21. A state in which the system can allocate resources without entering deadlock
22. Condition where a process waits indefinitely without getting CPU or resources
28. CPU scheduling algorithm that selects process with highest priority
30. Total time a process spends waiting in the ready queue
32. CPU scheduling algorithm that selects process with the shortest burst time
11. Classic synchronization problem involving shared chopsticks
13. Classic synchronization problem involving producer and consumer
15. Scheduling where a running process cannot be interrupted
18. CPU scheduling algorithm using fixed time quantum
23. Technique to prevent starvation by gradually increasing process priority
24. A program in execution with its own resources and state
25. Data structure that stores all information about a process

A crossword puzzle grid with 15 numbered squares. The grid is filled with letters, and the words are: 1. SILENT, 2. PNOPIE, 3. BOY, 4. KROX, 5. KOUT, 6. COUP, 7. SET, 8. YMO, 9. COUP, 10. SET, 11. YMO, 12. COUP, 13. SET, 14. YMO, 15. COUP.

33. Problem where producers generate data and consumers use it

36. Goal of CPU scheduling to minimize time between submission and completion

37. Condition where only one process can access a resource at a time

40. Total time taken from process submission to completion

41. CPU scheduling algorithm that serves processes in order of arrival

44. Condition where resources cannot be forcibly taken from a process

45. Module that gives CPU control to the process selected by scheduler

46. Different conditions of a process such as new, ready, running, waiting, terminated

26. Graph used to represent processes and resource allocation

27. Synchronization tool that provides mutual exclusion

29. A state from which deadlock might occur if resources are allocated

31. System call that replaces a process with a new program

34. System call used to create a new process

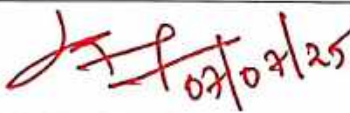
35. Method of identifying and resolving deadlocks in a system

38. Algorithm used to avoid deadlock by checking safe states

39. Part of a program where shared resources are accessed

42. Condition where a circular chain of processes each waits for a resource

43. Queue where processes wait for CPU allocation

Total marks (out of 20) :	10
Faculty signature with date :	 02/02/25

Simulation and Comparative Analysis of CPU Scheduling Algorithms



Project Description:

This project implements a functional simulator to compare the performance of various CPU scheduling algorithms. It evaluates their performance using key metrics such as throughput, fairness, and CPU efficiency.



Objectives:

1. Simulate multiple CPU scheduling algorithms.
2. Measure:
 - Throughput: Number of processes completed per unit time.
 - Fairness: Equal opportunity for processes to access CPU.
 - Efficiency: CPU utilization and idle time.
3. Generate data-driven charts to visualize performance differences.



Scheduling Algorithms Implemented:

1. First-Come First-Served (FCFS)
2. Shortest Job First (SJF)
3. Round Robin (RR)

4. Priority Scheduling

5. (Optional advanced): Multilevel Feedback Queue

How It Works:

- The simulator takes a list of processes as input.
- Each process has attributes:
 - PID, arrival_time, burst_time, and priority (if applicable).
- The simulation runs each algorithm on the same dataset.
- For each run, the program records:
 - Completion time of each process
 - Total idle time
 - Turnaround time and waiting time
 - Number of context switches

Performance Metrics:

- Throughput: $\text{Total completed processes} / \text{Total simulation time}$
- CPU Efficiency: $1 - (\text{idle_time} / \text{total_time})$
- Fairness: Standard deviation or variance of waiting times

Technologies Used:

- Language: C

Expected Output:

1. Tabular output with detailed process stats for each algorithm

Comparative graphs:

- Throughput per algorithm
 - Efficiency over time
 - Waiting time distribution
3. Console summary of best-performing algorithm per metric

CODING:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
struct Process {
```

```
    int pid, at, bt, ct, wt, tat, prio;
```

```
};
```

```
// Function to print Gantt Chart
```

```
void printGanttChart(int timeline[], int processes[], int n) {
```

```
    printf("\nGantt Chart:\n|");
```

```
    for (int i = 0; i < n; i++) {
```

```
        printf(" P%d |", processes[i]);
```

```
    }
```

```
    printf("\n");
```

```
    printf("%d", 0); // Start time
```

```
    for (int i = 0; i < n; i++) {
```

```
        printf(" %d", timeline[i]);
```

```
printf("\n");  
}
```

// Function to print performance metrics

```
void printMetrics(struct Process p[], int n) {
```

```
    float avgWT = 0, avgTAT = 0;
```

```
    printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");
```

```
    for (int i = 0; i < n; i++) {
```

```
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n", p[i].pid, p[i].at,  
p[i].bt,
```

```
        p[i].ct, p[i].tat, p[i].wt);
```

```
        avgWT += p[i].wt;
```

```
        avgTAT += p[i].tat;
```

```
    }
```

```
    printf("\nAverage TAT = %.2f", avgTAT / n);
```

```
    printf("\nAverage WT = %.2f\n", avgWT / n);
```

```
}
```

// First Come First Serve

```
void FCFS(struct Process p[], int n) {
```

```
printf("\n--- First Come First Serve (FCFS) ---\n");
```

```
int time = 0;
```

```
int ganttProc[n], ganttTime[n];
```

```
for (int i = 0; i < n; i++) {
```

```
    if (time < p[i].at) time = p[i].at;
```

```
    time += p[i].bt;
```

```
    p[i].ct = time;
```

```
    p[i].tat = p[i].ct - p[i].at;
```

```
    p[i].wt = p[i].tat - p[i].bt;
```

```
    ganttProc[i] = p[i].pid;
```

```
    ganttTime[i] = time;
```

```
}
```

```
printMetrics(p, n);
```

```
printGanttChart(ganttTime, ganttProc, n);
```

```
}
```

```
// Shortest Job First (Non-preemptive)
```

```
void SJF(struct Process p[], int n) {
```

```
    printf("\n--- Shortest Job First (SJF Non-Preemptive) ---\n");
```

```
    int completed = 0, time = 0, ganttProc[n], ganttTime[n], gi = 0;
```

```
int done[100] = {0};
```

```
while (completed < n) {
```

```
    int idx = -1, minBT = 1e9;
```

```
    for (int i = 0; i < n; i++) {
```

```
        if (!done[i] && p[i].at <= time && p[i].bt < minBT) {
```

```
            minBT = p[i].bt;
```

```
            idx = i;
```

```
        }
```

```
    }
```

```
    if (idx == -1) { time++; continue; }
```

```
    time += p[idx].bt;
```

```
    p[idx].ct = time;
```

```
    p[idx].tat = p[idx].ct - p[idx].at;
```

```
    p[idx].wt = p[idx].tat - p[idx].bt;
```

```
    ganttProc[gi] = p[idx].pid;
```

```
    ganttTime[gi++] = time;
```

```
    done[idx] = 1;
```

```
    completed++;
```

```
}
```

```
printMetrics(p, n);
```

```
printGanttChart(ganttTime, ganttProc, gi);
```

```
}
```

```
// Priority Scheduling (Non-preemptive)
```

```
void Priority(struct Process p[], int n) {
```

```
    printf("\n--- Priority Scheduling (Non-Preemptive) ---\n");
```

```
    int completed = 0, time = 0, ganttProc[n], ganttTime[n], gi = 0;
```

```
    int done[100] = {0};
```

```
    while (completed < n) {
```

```
        int idx = -1, minPrio = 1e9;
```

```
        for (int i = 0; i < n; i++) {
```

```
            if (!done[i] && p[i].at <= time && p[i].prio < minPrio) {
```

```
                minPrio = p[i].prio;
```

```
                idx = i;
```

```
            }
```

```
        }
```

```
        if (idx == -1) { time++; continue; }
```

```
        time += p[idx].bt;
```

```
        p[idx].ct = time;
```

```
        p[idx].tat = p[idx].ct - p[idx].at;
```

```
        p[idx].wt = p[idx].tat - p[idx].bt;
```

```

    ganttProc[gi] = p[idx].pid;
    ganttTime[gi++] = time;
    done[idx] = 1;
    completed++;
}

printMetrics(p, n);
printGanttChart(ganttTime, ganttProc, gi);
}

```

// Round Robin

```

void RoundRobin(struct Process p[], int n, int tq) {
    printf("\n--- Round Robin Scheduling (TQ=%d) ---\n", tq);
    int time = 0, remaining = n;

    int ganttProc[100], ganttTime[100], gi = 0;
    int bt[n], ct[n];
    for (int i = 0; i < n; i++) bt[i] = p[i].bt;

    int done = 0;
    while (done < n) {
        int executed = 0;
        for (int i = 0; i < n; i++) {

```

```
if (bt[i] > 0 && p[i].at <= time) {
```

```
    executed = 1;
```

```
    ganttProc[gi] = p[i].pid;
```

```
    if (bt[i] > tq) {
```

```
        time += tq;
```

```
        bt[i] -= tq;
```

```
    } else {
```

```
        time += bt[i];
```

```
        bt[i] = 0;
```

```
        ct[i] = time;
```

```
        done++;
```

```
    }
```

```
    ganttTime[gi++] = time;
```

```
}
```

```
}
```

```
if (!executed) time++;
```

```
}
```

```
for (int i = 0; i < n; i++) {
```

```
    p[i].ct = ct[i];
```

```
    p[i].tat = p[i].ct - p[i].at;
```

```
p[i].wt = p[i].tat - p[i].bt;  
}
```

```
printMetrics(p, n);  
printGanttChart(ganttTime, ganttProc, gi);  
}
```

```
int main() {
```

```
    int n, tq;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    struct Process p[n], copy1[n], copy2[n], copy3[n];
```

```
    for (int i = 0; i < n; i++) {
```

```
        printf("Enter Arrival Time, Burst Time, Priority for P%d: ", i  
+ 1);
```

```
        p[i].pid = i + 1;
```

```
        scanf("%d %d %d", &p[i].at, &p[i].bt, &p[i].prio);
```

```
    }
```

```
    // Keep copies for each algorithm
```

memcpy(copy1, p, sizeof(p));

memcpy(copy2, p, sizeof(p));

memcpy(copy3, p, sizeof(p));

FCFS(p, n);

SJF(copy1, n);

Priority(copy2, n);

printf("\nEnter Time Quantum for RR: ");

scanf("%d", &tq);

RoundRobin(copy3, n, tq);

return 0;

}

OUTPUT:

Enter number of processes: 5

Enter Arrival Time, Burst Time, Priority for P1: 0 7 4

Enter Arrival Time, Burst Time, Priority for P2: 2 9 3

Enter Arrival Time, Burst Time, Priority for P3: 1 6 2

Enter Arrival Time, Burst Time, Priority for P4: 3 8 1

Enter Arrival Time, Burst Time, Priority for P5: 4 5 5

--- First Come First Serve (FCFS) ---

PID	AT	BT	CT	TAT	WT
-----	----	----	----	-----	----

P1	0	7	7	7	0
----	---	---	---	---	---

P2	2	9	16	14	5
----	---	---	----	----	---

P3	1	6	22	21	15
----	---	---	----	----	----

P4	3	8	30	27	19
----	---	---	----	----	----

P5	4	5	35	31	26
----	---	---	----	----	----

Average TAT = 20.00

Average WT = 13.00

Gantt Chart:

| P1 | P2 | P3 | P4 | P5 |

0 7 16 22 30 35

--- Shortest Job First (SJF Non-Preemptive) ---

PID	AT	BT	CT	TAT	WT
-----	----	----	----	-----	----

P1	0	7	7	7	0
----	---	---	---	---	---

P2	2	9	35	33	24
----	---	---	----	----	----

P3	1	6	18	17	11
----	---	---	----	----	----

P4	3	8	26	23	15
----	---	---	----	----	----

P5	4	5	12	8	3
----	---	---	----	---	---

Average TAT = 17.60

Average WT = 10.60

Gantt Chart:

| P1 | P5 | P3 | P4 | P2 |

0 7 12 18 26 35

--- Priority Scheduling (Non-Preemptive) ---

	PID	AT	BT	CT	TAT	WT
--	-----	----	----	----	-----	----

	P1	0	7	7	7	0
--	----	---	---	---	---	---

	P2	2	9	30	28	19
--	----	---	---	----	----	----

	P3	1	6	21	20	14
--	----	---	---	----	----	----

	P4	3	8	15	12	4
--	----	---	---	----	----	---

	P5	4	5	35	31	26
--	----	---	---	----	----	----

Average TAT = 19.60

Average WT = 12.60

Gantt Chart:

| P1 | P4 | P3 | P2 | P5 |

0 7 15 21 30 35

Enter Time Quantum for RR: 4

--- Round Robin Scheduling (TQ=4) ---

	PID	AT	BT	CT	TAT	WT
--	-----	----	----	----	-----	----

	P1	0	7	23	23	16
--	----	---	---	----	----	----

UM
Process

P2 2 9 35 33 24

P3 1 6 29 28 22

P4 3 8 33 30 22

P5 4 5 34 30 25

Average TAT = 28.80

Average WT = 21.80

Gantt Chart:

| P1 | P2 | P3 | P4 | P5 | P1 | P2 | P3 | P4 | P5 | P2 |

0 4 8 12 16 20 23 27 29 33 34 35

=== Code Execution Successful ===

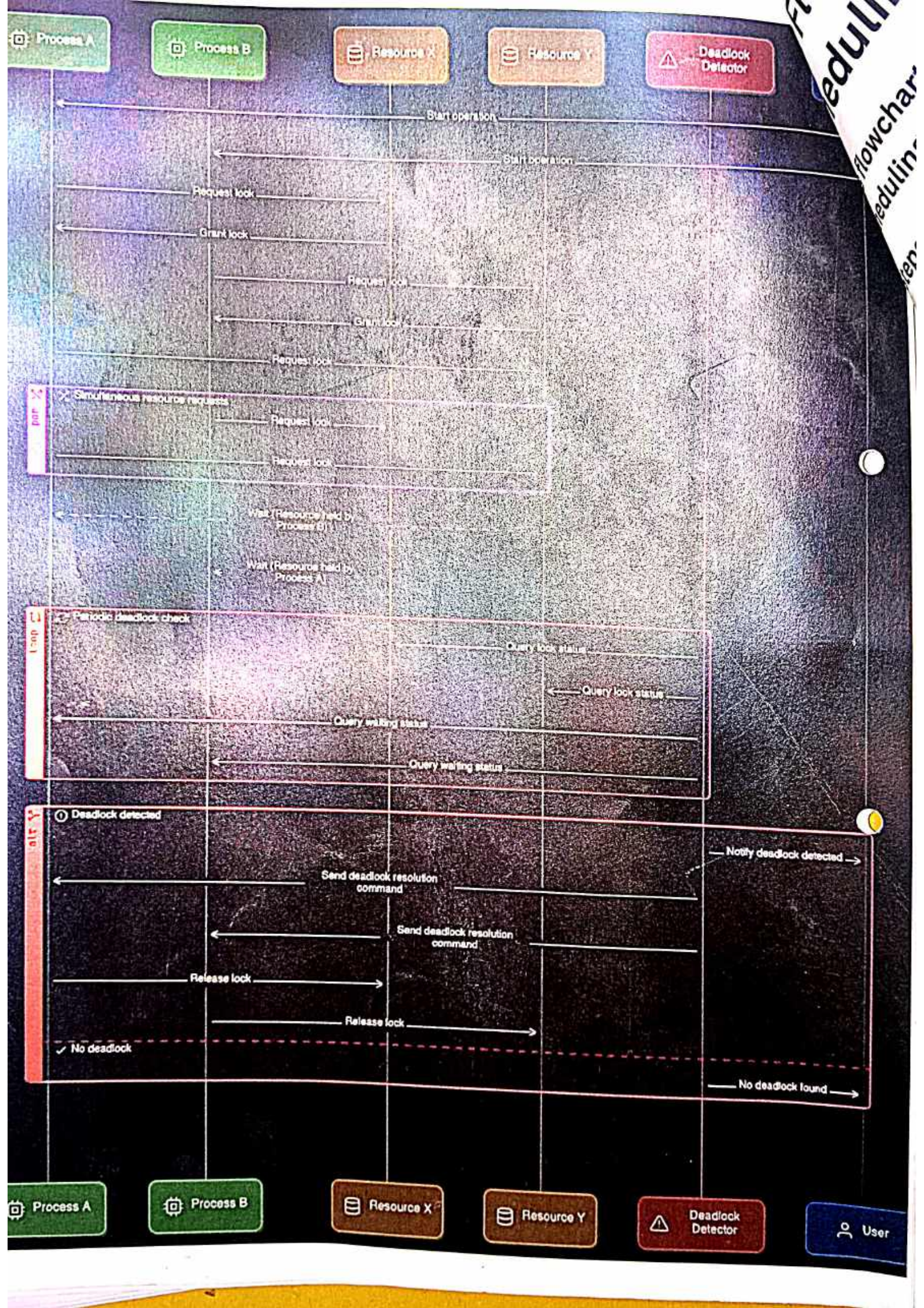
JML Sequence Diagram:

Process Interaction & Deadlock Detection

This diagram captures the dynamic behavior of two processes (A and B) trying to access shared resources (X and Y). Here's what it shows:

Key Points:

- Processes A and B both start operations and request locks on different resources.
 - Resource X is locked by A, and Resource Y is locked by B.
 - Later, A requests Y and B requests X, leading to a circular wait—a classic deadlock.
 - A Deadlock Detector periodically checks for this by querying lock and wait statuses.
 - Upon detecting a deadlock, it sends resolution commands to release the resources.
 - The deadlock is then resolved, and operations continue.
- 🧠 This illustrates a real-world concurrency issue and how a system might detect and recover from a deadlock using active monitoring.



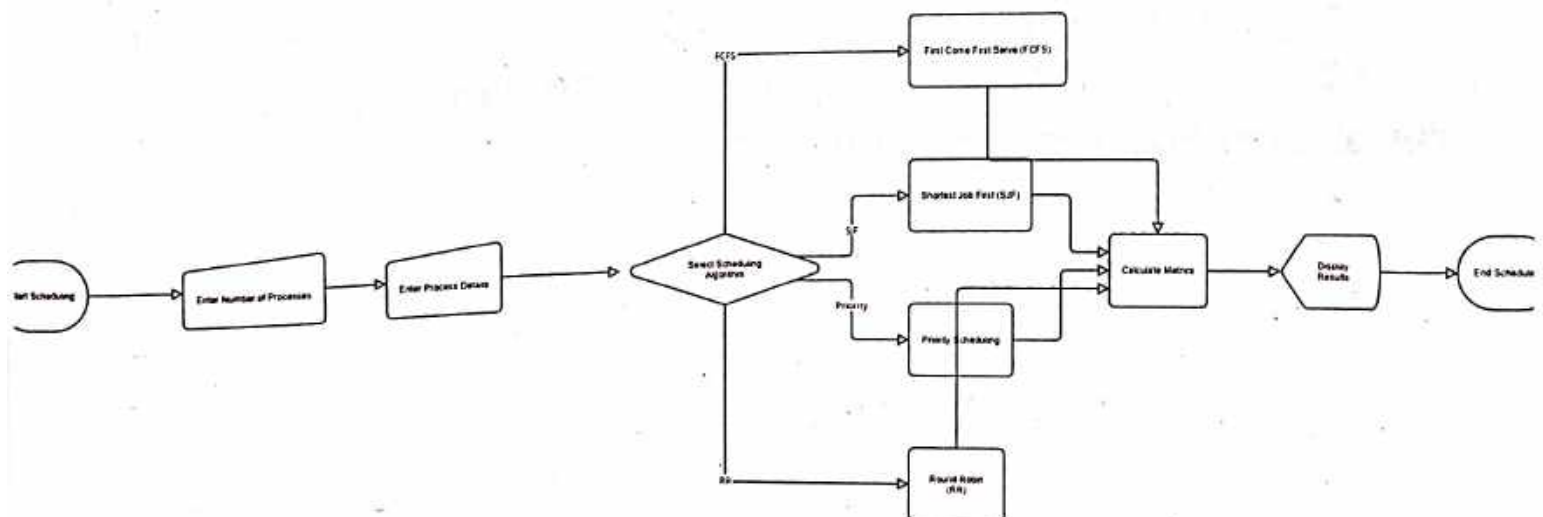
Flowchart: Process Scheduling Algorithm

This flowchart describes how a system handles CPU scheduling:

Steps:

1. Start scheduling.
2. Input the number of processes and their details.
3. Choose a scheduling algorithm:
 - FCFS (First Come First Serve)
 - SJF (Shortest Job First)
 - Priority Scheduling
 - RR (Round Robin)
4. Based on the chosen algorithm, calculate relevant metrics (like turnaround time, waiting time).
5. Display results and end the scheduling process.

🧠 This helps visualize how different scheduling algorithms work and how they impact process execution order.



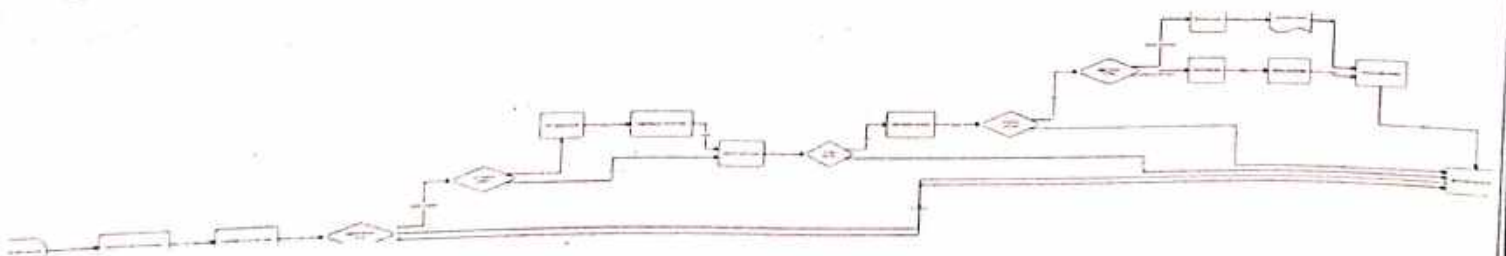
● 3. Flowchart: Deadlock Detection & Recovery

This flowchart shows how the system handles resource allocation and deadlock management:

Workflow:

- The system begins by allocating resources to processes.
- It constructs a Resource Allocation Graph to monitor dependencies.
- When a process requests a resource:
 - The system updates the graph.
 - It checks for cycles (which indicate a deadlock).
- If a deadlock is found:
 - Deadlock recovery is triggered:
 - Resources may be forcibly released.
 - Affected processes may be restarted or rolled back.
- If no deadlock, the system waits for new requests.

🧠 This diagram emphasizes the importance of cycle detection in graphs for deadlock handling and shows both prevention and resolution paths.



3. Records of Quality Assessment in Quiz, Seminar, Concept Map / Cheat Sheet, Crossword Puzzle, Project-Based Learning, and Case Study

A. Overview

The department adopts multiple continuous assessment tools beyond written tests and assignments to evaluate students' learning performance, creativity, problem-solving ability, and real-world application of knowledge. Each tool is designed with clear Course Outcome (CO) mapping and aligned with Bloom's Taxonomy (BTL) to ensure comprehensive coverage of cognitive levels — from *understanding* to *creation*.

B. Assessment Components and Process

Component	Purpose	Mode of Evaluation	Evaluation Criteria / Rubrics
1. Quiz / Viva Voce	To assess conceptual understanding and recall of core topics	Conducted online/offline; individual assessment	Accuracy, response time, comprehension
2. Seminar / Presentation	To develop communication, critical thinking, and research orientation	Individual or group presentation on emerging topics	Content depth, presentation clarity, analysis, creativity
3. Concept Map / Cheat Sheet	To promote visual learning and synthesis of complex concepts	Prepared by students summarizing unit topics	Clarity of concept flow, hierarchy, interconnections, originality
4. Crossword Puzzle / Gamified Assessment	To test quick recall and active learning in a fun, engaging manner	Conducted in class or through digital tools	Accuracy, time efficiency, engagement
5. Project-Based Learning (PBL)	To apply interdisciplinary knowledge to solve real-life or simulated problems	Team-based projects related to course content	Problem definition, design approach, implementation, innovation, teamwork
6. Case Study / Scenario Analysis	To apply theoretical knowledge to real-world engineering contexts	Case discussions, report analysis, solution presentation	Problem identification, analytical reasoning, justification of solution, presentation quality

C. Quality Assurance and Continuous Improvement

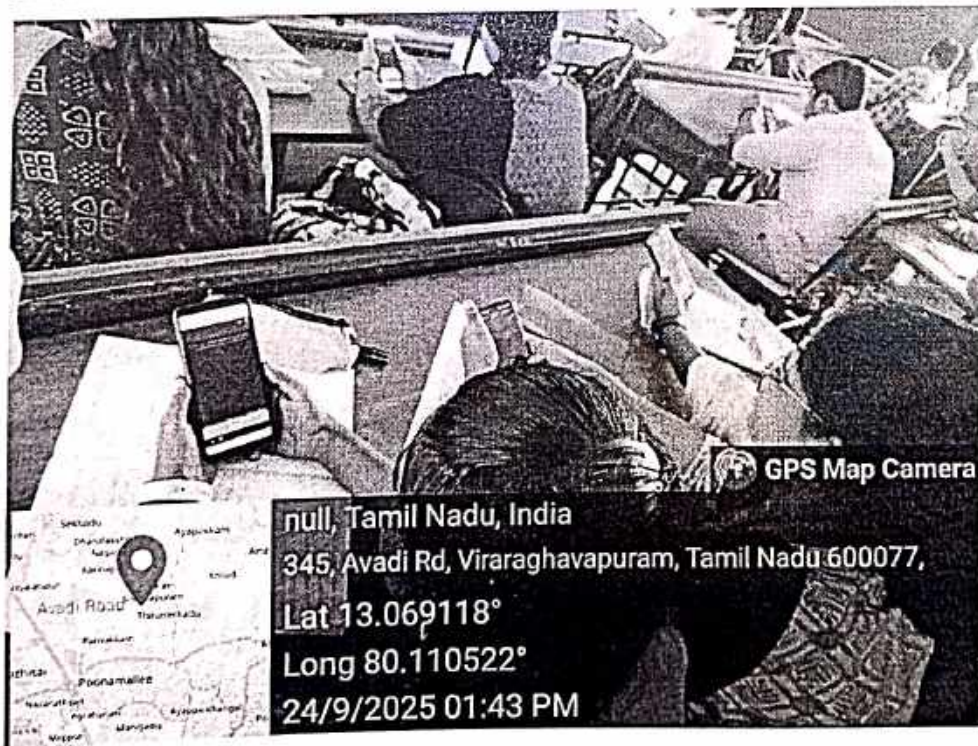
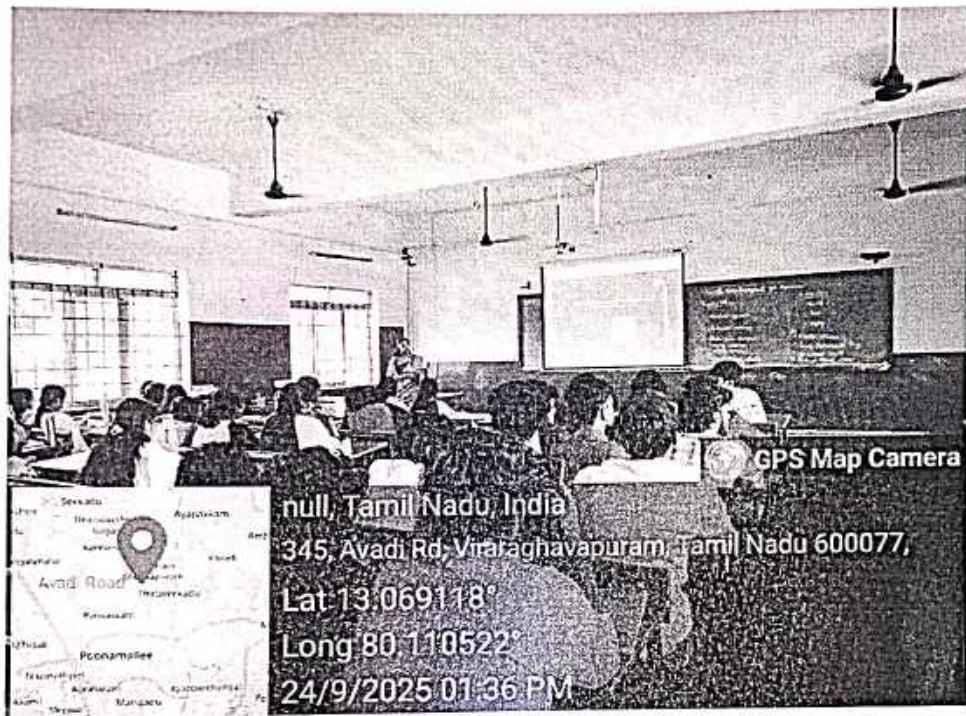
Parameter	Assessment Method	Improvement Action
CO Alignment	Verified by Course Coordinator	Revised CO-BTL mapping if imbalance found
Evaluation Transparency	Common rubrics and peer verification	Rubrics updated and standardized

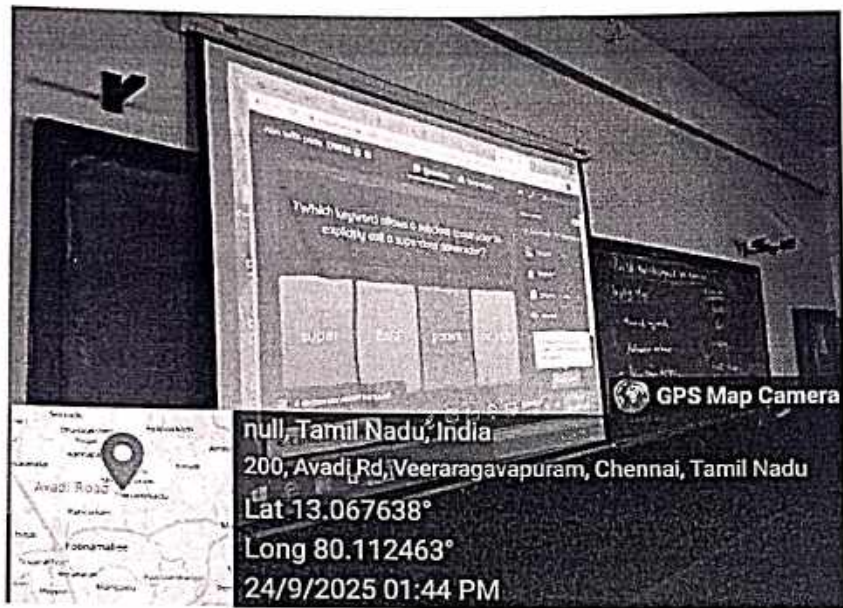
Student Feedback	Collected post-activity	Incorporated in next academic cycle
Attainment Review	Based on marks distribution and outcomes	Improved activity design for next semester

IT 3304 OBJECT ORIENTED PROGRAMMING

Link: <https://wayground.com/admin/reports/68c8353ef14cdd2c2e7c1825/players>

A quiz was conducted for the course IT3304 – Object Oriented Programming, covering Unit I to Unit III topics. The activity aimed to assess students' understanding of Java fundamentals, Object-Oriented concepts, inheritance principles, Exception and input output operations.







SUBJECT: IT3301/OBJECT ORIENTED PROGRAMMING
CLASS/SEM: II CSE /III

QUIZ RUBRIC – IT3304: OBJECT ORIENTED PROGRAMMING

Assessment Component: Quiz (15 Questions – 15 Marks)

CO Coverage: CO1, CO2, CO3

Bloom's Level: L1 – Remember, L2 – Understand, L3 – Apply, L4 – Analyse

Rubric Description

Performance Level	Marks Range (out of 20)	Performance Descriptor	Qualitative Description
Excellent (Level 3)	17 – 20	Demonstrates comprehensive understanding of all three units; applies OOP principles, inheritance, interfaces, exception handling, and multithreading correctly.	Student answers all conceptual, output-based, and application questions accurately; shows clarity in logic and syntax.
Good (Level 2)	13 – 16	Demonstrates good understanding with minor conceptual or syntactical errors.	Able to apply and analyze concepts but may miss a few advanced logic-based questions.
Average (Level 1)	9 – 12	Demonstrates basic understanding of core Java and OOP concepts.	Able to recall facts and definitions but struggles with applied or analytical questions.
Below Average (Level 0)	≤ 8	Demonstrates limited understanding or insufficient application of Java fundamentals.	Errors in basic syntax, misconceptions in inheritance or threading; poor logical interpretation.

Unit-Wise Question Distribution

Unit	Topic Focus	No. of Questions	CO Mapped	Bloom's Level
UNIT I	OOP principles, Java features, classes, constructors, strings, packages	5	CO1	L1, L2
UNIT II	Inheritance, interfaces, I/O streams, abstract/final classes	5	CO2	L2, L3
UNIT III	Exception handling, threads, synchronization, inter-thread communication	5	CO3	L3, L4

Bloom's Taxonomy Distribution

Cognitive Level	No. of Questions	Percentage
L1 – Remembering	3	20%
L2 – Understanding	4	27%
L3 – Applying	5	33%
L4 – Analyzing	3	20%

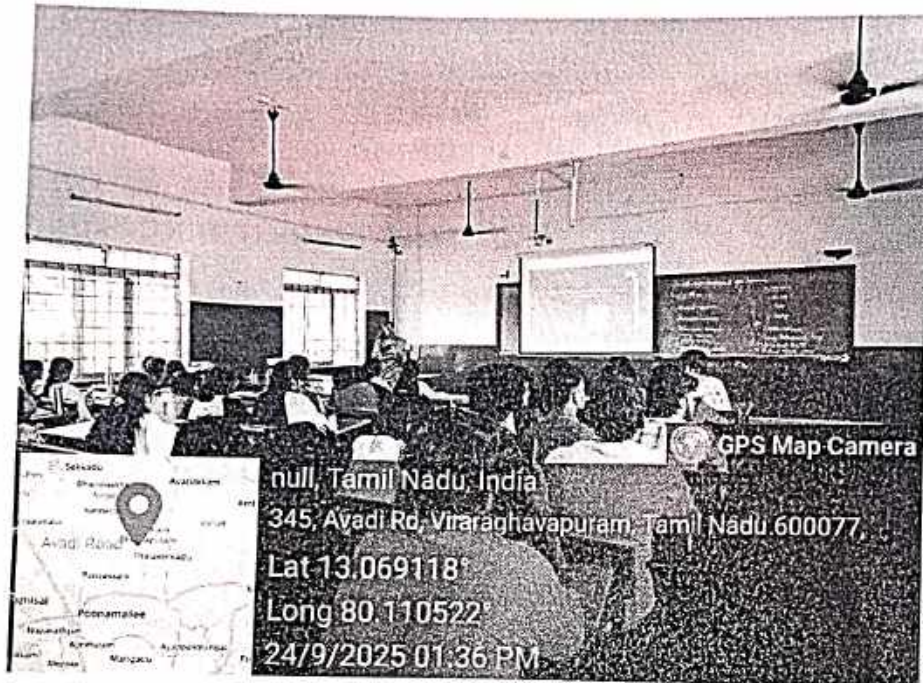
Overall Assessment Interpretation

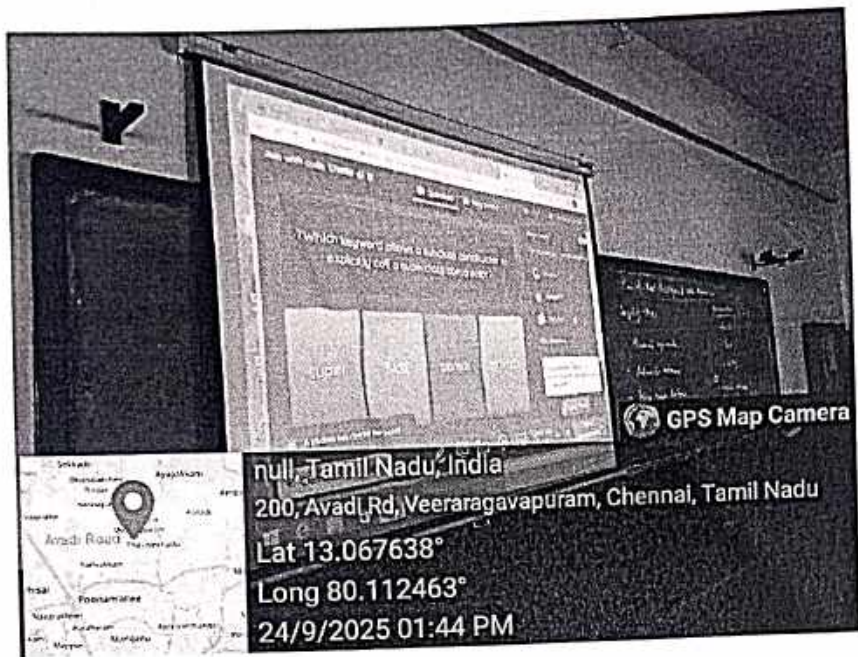
Parameter	Student Result	Remarks
Accuracy		
CO Attainment Level		

IT 3304 OBJECT ORIENTED PROGRAMMING

Link: <https://wayground.com/admin/reports/68c8353ef14cdd2c2e7c1825/players>

A quiz was conducted for the course IT3304 – Object Oriented Programming, covering Unit I to Unit III topics. The activity aimed to assess students' understanding of Java fundamentals, Object-Oriented concepts, inheritance principles, Exception and input output operations.





IT 3304 OBJECT ORIENTED PROC

Seminar

Harshini of II CSE D delivered a presentation on controls in Javar X, explaining its key concepts, features, and applications in GUI development.



S.A ENGINEERING COLLEGE, CHENNAI-77

Department of Computer Science & Engineering

Rubrics for Concept map

Register Number:

Subject Code/Subject Title:

Degree/Branch/Class:

Academic Year/Semester:

Date of submission:

Evaluated by:

Criteria	Excellent (4)	Good (3)	Satisfactory (2)	Needs Improvement (1)
Understanding of Concepts	Demonstrates thorough understanding of all concepts; no errors	Shows good understanding; minor errors	Shows basic understanding but some errors	Shows little understanding; many errors
Organization & Structure	Concepts are logically organized; hierarchy is clear and relationships are precise	Mostly logical organization; minor inconsistencies	Some logical order; relationships are unclear	Poorly organized; relationships are missing or incorrect
Connections & Relationships	All relevant connections are present; cross-links show deep understanding	Most connections are correct; few missing	Some connections present; cross-links are minimal	Connections missing; relationships mostly incorrect
Clarity & Labeling	All nodes and links are clearly labeled; terms are accurate	Most nodes and links labeled; minor inaccuracies	Some labels missing or unclear	Many labels missing; unclear or incorrect
Creativity & Visual Appeal	Highly creative layout; visually appealing; easy to follow	Layout is neat; mostly clear visually	Layout is acceptable but cluttered	Layout is messy; hard to follow

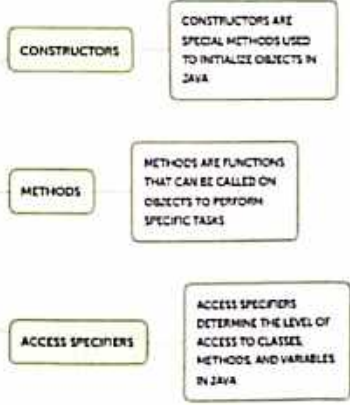
Grade secured: Excellent/Good/Satisfactory/Unsatisfactory

FOUNDATIONS AND ADVANCED CONCEPTS OF JAVA PROGRAMMING

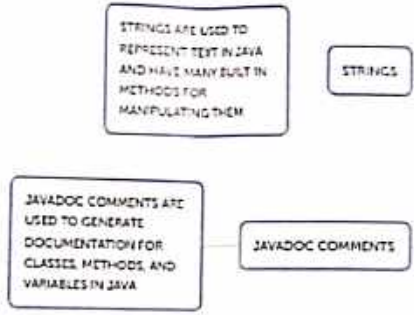
1. OBJECT ORIENTED PROGRAMMING



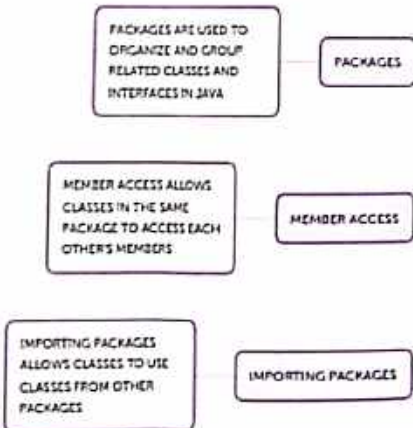
2. DEFINING CLASSES IN JAVA



4. STRINGS AND JAVADOC COMMENTS



3. PACKAGES



S.A ENGINEERING COLLEGE, CHENNAI-77

Department of Computer Science & Engineering

Rubrics for Cheat sheet

Register Number:

Subject Code/Subject Title:

Degree/Branch/Class:

Academic Year/Semester:

Date of submission:

Evaluated by:

Criteria	Excellent (4)	Good (3)	Satisfactory (2)	Needs Improvement (1)
Content Accuracy	All information is correct and relevant;.no errors	Most information is correct. minor errors	Some information correct. some errors	Most information incorrect or irrelevant
Completeness	Covers all key concepts, formulas, and examples needed	Covers most key concepts;minor gaps	Covers some key concepts. significant gaps	Covers very few key concepts; incomplete
Organization & Layout	Very well organized. easy to locate information quickly	Organized; mostly easy to navigate	Some organization. hard to locate some information	Poorly organized. difficult to use
Conciseness & Clarity	Information is concise, clear, and easy to read	Mostly concise and clear. minor clutter	Somewhat cluttered or unclear	Very cluttered or confusing
Usefulness / Practicality	Extremely helpful for quick reference.Formulas, examples, and notes are practical	Generally helpful. minor missing items	Somewhat helpful. missing many important items	Not useful; missing essential content

Grade secured: Excellent/Good/Satisfactory/Unsatisfactory

Java Object Oriented Programming Language Cheatsheet (IT3304)

OBJECT-ORIENTED CONCEPTS

- ENCAPSULATION, INHERITANCE, POLYMORPHISM, ABSTRACTION
- JAVA FEATURES → SIMPLE, SECURE, PORTABLE, ROBUST, MULTITHREADED
- ACCESS SPECIFIERS → PUBLIC, PRIVATE, PROTECTED, DEFAULT
- THIS → CURRENT OBJECT / SUPER → PARENT CLASS
- STATIC VS INSTANCE MEMBERS

CODE - CLASSES & OBJECTS

```

class Student {
    int id; String name;
    void display() { System.out.println("ID: " + id + " Name: " + name); }
    public static void main(String[] args) {
        Student s1 = new Student();
        s1.setId(123); s1.setName("Alice"); s1.display();
    }
}
    
```

PACKAGES

- ORGANIZE CLASSES INTO GROUPS
- PACKAGE KEYWORD, USE IMPORT

CODE - PACKAGES

```

package mypack;
public class Message {
    public void show() { System.out.println("Hello from package!"); }
}
    
```

IMPORT mypack.Message;

```

class TestPackage {
    public static void main(String[] args) {
        Message m = new Message(); m.show();
    }
}
    
```

EXCEPTION HANDLING

- TRY-CATCH-FINALLY
- THROW / THROWS
- CHECKED VS UNCHECKED

CODE - EXCEPTION HANDLING

```

class Test {
    public static void main(String[] args) {
        try { int a = 10/0; }
        catch (ArithmeticException e) { System.out.println("Error: " + e); }
        finally { System.out.println("Finally block executed"); }
    }
}
    
```

INHERITANCE & POLYMORPHISM

- EXTENDS → SINGLE INHERITANCE
- ABSTRACT CLASS → CANNOT BE INSTANTIATED
- FINAL → STOP INHERITANCE/OVERLOADING

CODE - INHERITANCE

```

class Animal { void eat() { System.out.println("Animal eating"); } }
class Dog extends Animal { void bark() { System.out.println("Barking"); } }
class Test {
    public static void main(String[] args) {
        Dog d = new Dog(); d.bark();
    }
}
    
```

CODE - ABSTRACT CLASS & POLYMORPHISM

```

abstract class Shape { abstract void draw(); }
class Circle extends Shape { void draw() { System.out.println("Circle"); } }
class Square extends Shape { void draw() { System.out.println("Square"); } }
class Test {
    public static void main(String[] args) {
        Shape s = new Circle(); s.draw();
        s = new Square(); s.draw();
    }
}
    
```

STRINGS

- STRING → IMMUTABLE
- STRINGBUFFER → MUTABLE, THREAD-SAFE
- STRINGBUILDER → MUTABLE, FAST

CODE - STRING & STRINGBUILDER

```

class TestString {
    public static void main(String[] args) {
        String s = "Hello";
        System.out.println(s.toUpperCase());
        StringBuffer sb = new StringBuffer("Hello");
        sb.append(" World");
        System.out.println(sb);
    }
}
    
```

GENERIC & COLLECTIONS

- GENERICS → TYPE SAFETY, REUSABILITY
- COLLECTIONS → LIST, SET, QUEUE, MAP

CODE - GENERICS EXAMPLE

```

class Box<T> {
    T value;
    Box(T value) { this.value = value; }
    void show() { System.out.println(value); }
}
class Test {
    public static void main(String[] args) {
        Box<Integer> b = new Box<>(100);
        b.show();
    }
}
    
```

INTERFACES (MULTIPLE INHERITANCE)

- INTERFACES → MULTIPLE INHERITANCE IN JAVA

CODE - INTERFACES

```

interface A { void showA(); }
interface B { void showB(); }
class Demo implements A, B {
    public void showA() { System.out.println("Show A"); }
    public void showB() { System.out.println("Show B"); }
    public static void main(String[] args) {
        Demo d = new Demo(); d.showA(); d.showB();
    }
}
    
```

FILE HANDLING

- FILEREADER, FILEWRITER, BUFFEREDREADER

CODE - FILE OPERATIONS

```

import java.io.*;
class FileOps {
    public static void main(String[] args) {
        try {
            FileWriter fw = new FileWriter("test.txt");
            fw.write("Hello file"); fw.close();
            BufferedReader br = new BufferedReader(new FileReader("test.txt"));
            System.out.println("Read: " + br.readLine());
            br.close();
        } catch (Exception e) { System.out.println(e); }
    }
}
    
```

MULTITHREADING

- EXTENDS THREAD OR IMPLEMENT RUNNABLE
- METHODS → START(), SLEEP(), JOIN()
- SYNCHRONIZATION PREVENTS RACE CONDITIONS

CODE - MULTITHREADING

```

class MyThread extends Thread {
    public void run() {
        for (int i = 1; i <= 5; i++) System.out.println(i);
    }
}
class Test {
    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        t1.start();
    }
}
    
```